**TRƯỜNG ĐẠI HỌC GIAO THÔNG VẬN TẢI**
**PHÂN HIỆU TẠI THÀNH PHỐ HỒ CHÍ MINH**
**KHOA KỸ THUẬT CÔNG NGHỆ**

# BÀI TẬP LỚN
# CẤU TRÚC DỮ LIỆU VÀ GIẢI THUẬT

# HỆ THỐNG XẾP HÀNG KHÁM BỆNH VÀ CẤP CỨU BỆNH VIỆN

## Đề tài ĐT02 - Nhóm 6

Giảng viên hướng dẫn: Trần Thị Dung


| STT | Họ và tên | MSSV |
|---|---|---|
| 1 | Nguyễn Vinh Quang | 6651020070 |
| 2 | Nguyễn Tiến Thành | 6651020080 |
| 3 | Tiết Trung Tín (Nhóm trưởng) | 6651020087 |
| 4 | Hồ Quốc Thịnh | 6651020083 |
| 5 | Lê Thành Sum | 6651020076 |


**TP. Hồ Chí Minh, tháng 11 năm 2026**

# Mục lục

**Đối chiếu yêu cầu YC01-YC04 và trạng thái thực tế** 4

**Lời mở đầu** 5

**Nghiệp vụ và phân tích bài toán** 6

Bối cảnh và mục tiêu . . . 6
Tác nhân, đầu vào và đầu ra . . . 6
Ràng buộc và giả định . . . 6
Ca sử dụng chính . . . 6

**Thiết kế cấu trúc dữ liệu và giải thuật** 7

Mô hình bệnh nhân và toán tử so sánh . . . 7
Queue bằng mảng vòng . . . 7
Priority Queue bằng danh sách liên kết . . . 7
Deque và vòng đời bộ nhớ . . . 7
Điều phối FIFO và ưu tiên . . . 8
Chuyển khoa cấp cứu . . . 8
Ước tính thời gian chờ . . . 8

**Cài đặt chương trình C++** 9

Tổ chức tệp . . . 9
Kiểm tra dữ liệu đầu vào . . . 9
Lưu trữ và nạp CSV . . . 9
Lỗi và khả năng phục hồi . . . 9

**Đánh giá độ phức tạp và kiểm thử** 10

Phương pháp kiểm thử . . . 10
Kịch bản thực nghiệm tái lập . . . 10
Các ca lỗi đáng chú ý . . . 10
Giới hạn kiểm chứng . . . 10

**Phân tích bất biến và tình huống biên** 11

Bất biến của Queue mảng vòng . . . 11

Bất biến của Priority Queue và Deque . . . 11
Giữ trạng thái nhất quán khi gọi và chuyển khoa . . . 11
Biên của ước tính thời gian chờ . . . 12
Giới hạn của việc lưu CSV và khả năng mở rộng . . . 12

**Kết luận và hướng phát triển** **13**

Bảng tổng hợp kiểm thử và độ phức tạp . . . 13

**Tài liệu tham khảo và minh chứng** **14**

**Phụ lục A. Mã nguồn C++ hiện hành** **15**

include/BenhNhan.h . . . 15
include/HangDoi.h . . . 15
include/HangUuTien.h . . . 17
include/HangDoiHaiDau.h . . . 18
include/HeThongBenhVien.h . . . 20
src/HeThong_TiepNha.cpp . . . 21
src/HeThong_GoiSo.cpp . . . 23
src/HeThong_ChuyenKhoa.cpp . . . 25
src/HeThong_ThongKe.cpp . . . 26
src/HeThong_LuuTru.cpp . . . 28
src/main.cpp . . . 32
tests/test_all.cpp . . . 38

# Đối chiếu yêu cầu YC01-YC04 và trạng thái thực tế

Báo cáo này chỉnh sửa bản PDF 50 trang ngày 01/11/2026 theo mã nguồn hiện có tại GitHub. Phần website đang nằm trong code/ và wed/; bản C++ console phục vụ nghiệm thu bài tập lớn nằm độc lập trong console/. Các trích đoạn mã ở phụ lục lấy trực tiếp từ console/ để tránh chép lại phiên bản cũ.

| Mã | Yêu cầu | Trạng thái có thể xác minh |
|---|---|---|
| YC01 | GitHub, commit riêng, README, nhánh và PR | Mã và README có trên main; lịch sử đóng góp từng người/PR vẫn cần nhóm đối chiếu. |
| YC02 | Jira theo dõi tiến độ, người phụ trách, hạn và tỷ lệ | NNT-58 đến NNT-76 đã tạo, gán người phụ trách, trạng thái To Do; hạn và chứng cứ tiến độ phải cập nhật thật. |
| YC03 | Báo cáo LaTeX 40-50 trang | Nguồn console/bao_cao.tex có sẵn; PDF này ấn bản chỉnh sửa. Nhóm cần rà lại bản LaTeX chính thức và nguồn trích dẫn. |
| YC04 | C++ ổn định, kiểm thử, slide và demo | 65/65 ca kiểm thử đạt, demo console chạy; slide 10 trang có sẵn. Cần ghi video/ảnh nghiệm thu nhóm. |

Repository: https://github.com/tiettrungtinvn-create/NH-M-N-Y-T-Y/tree/main/console

Jira: https://nguyentienthanh11207.atlassian.net/jira/software/projects/NNT/list

# Lời mở đầu

Trong phòng khám, thứ tự tiếp nhận và thứ tự được gọi không phải lúc nào cũng trùng nhau. Người khám thông thường cần được phục vụ theo nguyên tắc đến trước phục vụ trước, trong khi người bệnh có mức khẩn cấp cao phải được đưa lên trước. Nếu chỉ dùng một mảng không có quy tắc ưu tiên, hệ thống dễ gọi sai; nếu chỉ dùng hàng đợi ưu tiên, tính công bằng của người cùng mức khẩn cấp không được diễn giải rõ. Đề tài này tổ chức các luồng bằng ba cấu trúc dữ liệu: Queue, Priority Queue và Deque. Mục tiêu là minh họa cách chọn cấu trúc phù hợp cho một nghiệp vụ cụ thể, sau đó kiểm chứng bằng chương trình C++ console và bộ kiểm thử tự động.

Phạm vi của mô hình là điều phối trong một phiên trực với dữ liệu mô phỏng. Chương trình không chẩn đoán, không thay thế quy trình phân loại cấp cứu chuyên môn và không xử lý thông tin bệnh nhân thực. Mỗi lượt khám được giả định dài mười phút nhằm trình bày phép tính thời gian chờ; số phút dự kiến có thể thay đổi trong thực tế. Các quyết định điều phối và cấu trúc dữ liệu được mô tả ở những chương tiếp theo.

# Nghiệp vụ và phân tích bài toán

## Bối cảnh và mục tiêu

Một lượt tiếp nhận cấp một mã bệnh nhân, tên, phòng khám, mức ưu tiên và số thứ tự tự động. Danh sách phòng hình thành khi có bệnh nhân đầu tiên đến phòng đó. Trong mỗi phòng, người bệnh thường chờ theo FIFO; người ưu tiên được sắp theo mức khẩn cấp giảm dần. Trường hợp chuyển sang khoa cấp cứu nhận mức 999, cao hơn mức 1-5 thông thường. Khi gọi theo phòng, chỉ lấy người đứng đầu phù hợp của phòng được chọn. Khi gọi tự động toàn hệ thống, so sánh ứng viên đứng đầu các phòng và lấy người có thứ tự phục vụ cao nhất.

Mục tiêu kiểm chứng được là: không mất phần tử khi hàng đợi quay vòng; người cùng độ ưu tiên được gọi theo số thứ tự đến; bệnh nhân đã gọi rời hàng chờ và xuất hiện trong lịch sử; chuyển khoa không làm đổi tổng số người đang chờ; lưu rồi nạp lại duy trì số thứ tự, hàng chờ và lịch sử.

## Tác nhân, đầu vào và đầu ra

Tác nhân sử dụng bản console là nhân viên tiếp nhận hoặc người vận hành demo. Đầu vào gồm mã số nguyên dương, tên và phòng không rỗng (tối đa 50 ký tự), mức ưu tiên từ 1 đến 5. Các thao tác sau đó là gọi tự động, gọi theo phòng hoặc mã, chuyển cấp cứu, hủy, tra cứu thời gian, xem bảng và lưu/nạp CSV. Đầu ra là thông báo thành công hoặc lý do thất bại, thứ tự gọi, bảng điện tử và file dữ liệu.

## Ràng buộc và giả định

Chương trình giới hạn 100 phần tử cho một hàng và 10 phòng. Menu khởi tạo hệ thống tối đa 50 bệnh nhân chờ; lớp nghiệp vụ có thể khởi tạo với sức chứa khác nhưng không vượt quá giới hạn mảng. Số thứ tự là tăng dần trong một bộ dữ liệu và tiếp tục sau khi nạp CSV. Lịch sử chỉ giữ 100 lượt gọi gần nhất. Một mã bệnh nhân không được có đồng thời ở hai hàng chờ. Độ ưu tiên đầu vào 1 biểu thị thường; 2-5 là ưu tiên; 999 chỉ do thao tác chuyển khoa cấp cứu tạo ra. Thời gian chờ tính trong phòng của bệnh nhân, không phải tổng thời gian toàn bệnh viện.

## Ca sử dụng chính

Luồng tiếp nhận: nhập dữ liệu, kiểm tra, phát số, tìm hoặc tạo phòng, thêm vào Queue hay Priority Queue, tăng bộ đếm. Luồng gọi: chọn ứng viên, lấy khỏi hàng, giảm số đang chờ, thêm vào Deque lịch sử, phát thông báo. Luồng chuyển cấp cứu: tìm bệnh nhân, kiểm tra khoa đích, lấy khỏi hàng cũ, cập nhật ưu tiên và phòng, chèn vào hàng ưu tiên của khoa đích. Luồng ước tính: đếm người đứng trước trong cùng phòng rồi nhân mười phút.

# Thiết kế cấu trúc dữ liệu và giải thuật

## Mô hình bệnh nhân và toán tử so sánh

BenhNhan chứa mã, tên, phòng, độ ưu tiên và số thứ tự. Toán tử < so mức ưu tiên trước; nếu bằng nhau, số thứ tự lớn hơn được xem là nhỏ hơn trong thứ tự phục vụ. Vì vậy một đối tượng được xem là ``lớn" khi có độ khẩn cấp cao hơn hoặc đến sớm hơn trong cùng mức. operator> dùng đối xứng; operator== so mã. Đây là tiêu chí thống nhất để danh sách ưu tiên và bộ chọn toàn hệ thống không mâu thuẫn.

a đứng trước b khi a có ưu tiên cao hơn; nếu cùng mức, STT của a nhỏ hơn STT của b.

trong đó u là độ ưu tiên, s là số thứ tự. Nếu a.s<b.s và cùng mức u, a đứng trước b theo FIFO.

## Queue bằng mảng vòng

HangDoi lưu mảng cố định a[100], vị trí đầu dau và số phần tử dem. Phần tử mới được gán tại (dau+dem)mod 100. Lấy đầu trả a[dau], sau đó tăng dau theo modulo 100 và giảm dem. Điều kiện đầy là dem==100. Cách này tái sử dụng các ô ở đầu mảng đã giải phóng mà không phải dịch toàn bộ dữ liệu mỗi lần lấy.

Thao tác thêm, lấy và xem đầu đều O(1). Xóa theo mã phải tìm rồi dịch các phần tử tiếp theo trong thứ tự logic nên O(n). Đây là đánh đổi phù hợp: luồng chính là thêm cuối và lấy đầu, còn hủy/gọi theo mã ít xảy ra hơn.

## Priority Queue bằng danh sách liên kết

HangUuTien là danh sách liên kết đơn có thứ tự. Khi thêm, duyệt đến vị trí mà bệnh nhân mới có thứ tự phục vụ cao hơn phần tử tiếp theo, rồi nối nút. Đầu danh sách luôn là người cần được gọi đầu tiên. Khi mức ưu tiên bằng nhau, số thứ tự đảm bảo người vào trước vẫn đứng trước. Xem và lấy đầu tốn O(1); thêm và xóa theo mã tốn O(n). Bộ giới hạn 100 phần tử từ chối chèn khi đầy, tránh tăng bộ nhớ không kiểm soát trong bài toán mô phỏng.

## Deque và vòng đời bộ nhớ

HangDoiHaiDau dùng danh sách liên kết đôi với con trỏ đầu/cuối. Nó lưu lịch sử gọi số: thêm lượt mới ở cuối; khi đủ 100 lượt, lấy lượt cũ nhất ở đầu. Hai thao tác này là O(1). Destructor xóa toàn bộ nút còn lại. Các lớp sở hữu con trỏ không cho sao chép nông để tránh giải phóng hai lần. Kiểm thử D01-D04 bao gồm trường hợp lấy phần tử duy nhất rồi thêm lại.

## Điều phối FIFO và ưu tiên

Một phòng có hai hàng: hàng ưu tiên và hàng thường. Nếu hàng ưu tiên có phần tử, gọi đầu hàng đó; nếu không, gọi đầu hàng thường. Khi gọi toàn hệ thống, chương trình duyệt các đầu hàng ưu tiên của các phòng để lấy ứng viên lớn nhất theo toán tử so sánh. Chỉ khi không phòng nào có bệnh nhân ưu tiên mới duyệt các đầu hàng thường và chọn STT nhỏ nhất. Cách chọn này không cần chuyển người giữa các phòng. Với tối đa 10 phòng, một lần chọn tốn O(P) với $P \leq 10$; việc lấy đầu hàng là O(1).

## Chuyển khoa cấp cứu

Thao tác chuyển kiểm tra mã tồn tại, khoa đích hợp lệ, phòng còn chỗ và không chuyển lặp một ca đã cấp cứu. Sau đó bệnh nhân được lấy khỏi hàng cũ, đặt phòng mới và mức 999, thêm vào Priority Queue đích. Tổng người đang chờ không thay đổi. STT cũ được giữ nguyên để các ca cấp cứu cùng mức vẫn tuân theo thứ tự tiếp nhận. Giới hạn cần giải thích: tên khoa có thể do người vận hành truyền vào; menu dùng tên mặc định Khoa Cap Cuu.

## Ước tính thời gian chờ

Với người ưu tiên, số người đứng trước là vị trí của họ trong hàng ưu tiên. Với người thường, số người đứng trước bằng toàn bộ số người ưu tiên trong phòng cộng vị trí của họ trong hàng thường. Công thức:

Thời gian chờ dự kiến = 10 phút × số bệnh nhân đứng trước trong cùng phòng.

Nếu không tìm thấy mã đang chờ, hàm trả -1. Mô hình này giả định một phòng phục vụ tuần tự, mỗi lượt mất đúng mười phút. Nó không dự báo thời gian khám thực, thời gian bác sĩ nghỉ hoặc người mới đến với mức khẩn cấp cao hơn.

# Cài đặt chương trình C++

## Tổ chức tệp

Nhóm include/ khai báo dữ liệu, hàng đợi và lớp nghiệp vụ. Nhóm src/ cài đặt theo công việc: tiếp nhận, gọi số, chuyển khoa, thống kê và lưu trữ; main.cpp xử lý tham số, menu và kịch bản demo. tests/test_all.cpp kiểm tra 65 trường hợp. Cách chia này cho phép từng thành viên đọc một phần nhỏ, biên dịch chung bằng Makefile và chứng minh ranh giới giữa cấu trúc dữ liệu với giao diện console.

## Kiểm tra dữ liệu đầu vào

Trước khi đổi trạng thái, hệ thống từ chối mã không dương, tên/phòng rỗng, chỉ toàn khoảng trắng, quá 50 ký tự, chứa dấu phẩy hoặc xuống dòng, độ ưu tiên ngoài 1-5, mã trùng và sức chứa đầy. Dấu phẩy bị cấm vì định dạng CSV tự cài đặt chưa có cơ chế escape. Menu nhập lại khi người dùng gõ sai kiểu; EOF được xử lý để lưu và thoát. Một lỗi nghiệp vụ trả false và chuỗi loiCuoi(), giúp người vận hành biết lý do chính xác.

## Lưu trữ và nạp CSV

File bắt đầu bằng CAREQUEUE,1, tiếp theo là STT đã cấp, các dòng phòng R, người chờ Q và lịch sử H. Khi nạp, chương trình đọc và kiểm tra định dạng, kiểu số, giới hạn, mã/STT trùng trước khi xóa trạng thái cũ. Điều này ngăn file hỏng làm hệ thống rơi vào trạng thái chỉ nạp một phần. Khi lưu, nội dung được ghi vào file tạm rồi đổi tên sang file đích. Trên Windows, nếu không thể đổi tên đè file cũ, chương trình xóa file đích trước khi đổi tên; trong khoảng này có rủi ro mất bản cũ nếu thao tác cuối thất bại. Do vậy bản demo không nên dùng cho dữ liệu bệnh án thực và cần sao lưu file CSV khi chạy nhiều phiên.

## Lỗi và khả năng phục hồi

Khi file khởi động bị hỏng, chương trình sao lưu sang tệp đuôi .loi và bắt đầu với hệ thống rỗng; người vận hành có thể xem bản sao để sửa thủ công. Khi nạp lại trong phiên, lỗi được báo và dữ liệu đang chạy được giữ nguyên. Mất quyền ghi hoặc đầy đĩa ném ngoại lệ; tầng menu bắt lỗi và hiển thị thông báo. Giới hạn: không khóa file, do đó không chạy hai tiến trình cùng sửa một CSV.

# Đánh giá độ phức tạp và kiểm thử

## Phương pháp kiểm thử

Bộ kiểm thử tự động ở tests/test_all.cpp dùng macro CHECK để đếm ca đạt. Mỗi nhóm ca tạo hệ thống riêng để trạng thái không rò từ ca trước. Nhóm Queue kiểm tra FIFO, đầy, quay vòng và xóa giữa; Priority Queue kiểm tra ưu tiên và FIFO cùng mức; Deque kiểm tra thêm/lấy hai đầu và con trỏ sau khi rỗng. Nhóm nghiệp vụ kiểm tra tiếp nhận, gọi, hủy, chuyển khoa, bảng và thời gian. Nhóm file kiểm tra roundtrip và file sai định dạng.

## Kịch bản thực nghiệm tái lập

Tại thư mục gốc dự án, chạy ./run.sh test trên Linux/macOS hoặc biên dịch tests/test_all.cpp cùng năm tệp src/HeThong_*.cpp trên Windows. Sau đó chạy ./run.sh -demo hoặc run.bat -demo. Kịch bản chi tiết và kết quả mong đợi nằm tại docs/KICH_BAN_DEMO.md. Khi chạy lại, cần phân biệt dữ liệu mẫu với dữ liệu đang lưu: -demo dùng file tạm; menu thường dùng data/benhvien.csv.

## Các ca lỗi đáng chú ý

Thêm bệnh nhân thứ 101 vào một hàng bị từ chối; không ghi đè phần tử cũ. Nhập mã trùng bị từ chối trước khi phát STT. Gọi khi hàng rỗng không giảm số lượng âm. Chuyển cấp cứu cùng bệnh nhân lần hai báo đã ở khoa cấp cứu. Nạp CSV với trùng mã hoặc độ ưu tiên 9 ném lỗi và giữ dữ liệu đang chạy. Bộ kiểm thử cũng kiểm tra file CRLF trên Windows. Các ca này hướng vào bất biến của từng cấu trúc thay vì chỉ kiểm tra màn hình in ra.

## Giới hạn kiểm chứng

Kết quả 65/65 là chạy chức năng trên môi trường đóng gói; kiểm tra rò rỉ bộ nhớ bằng AddressSanitizer/Valgrind phụ thuộc hệ điều hành và chưa được xác nhận trên mọi máy. Bộ kiểm thử không mô phỏng nhiều nhân viên cùng ghi file. Không có đo hiệu năng với hàng triệu phần tử do sức chứa thiết kế nhỏ. Nhóm cần ghi máy, hệ điều hành, compiler và ảnh kết quả thật vào bản PDF cuối cùng.

# Phân tích bất biến và tình huống biên

## Bất biến của Queue mảng vòng

Trong HangDoi, cặp (dau, dem) xác định toàn bộ phần tử hợp lệ. Bất biến là $0 \le dem \le 100$ và phần tử logic thứ i nằm tại a[(dau+i) mod 100] với $0 \le i < dem$. Vì vậy vị trí thêm tiếp theo là a[(dau+dem) mod 100], kể cả khi chỉ số vật lý đã quay về đầu mảng. Thao tác lay chỉ tăng dau và giảm dem sau khi đọc phần tử đầu. Nó không cần xóa ô cũ: ô đó nằm ngoài miền logic sau khi dem giảm. Đây là lý do mảng vòng vẫn đúng ngay cả khi các ô còn chứa bản ghi cũ.

Ca Q04 kiểm tra trường hợp vòng lại một cách trực tiếp: nạp đầy 100 bệnh nhân, lấy 30, thêm 30 người mới, rồi đọc phần tử logic cuối. Kết quả phải là bệnh nhân mới thứ 30 và kichThuoc vẫn là 100. Nếu cài đặt dùng chỉ số cuoi tăng mà không lấy modulo, ca này sẽ ghi ra ngoài mảng. Ca Q05-Q06 kiểm tra layTheoMa ở giữa hàng: các phần tử phía sau được dịch theo chỉ số logic, nhờ đó FIFO của những người còn lại không đổi.

## Bất biến của Priority Queue và Deque

HangUuTien giữ thứ tự từ đầu đến cuối theo quan hệ phục vụ: phần tử đứng trước không nhỏ hơn phần tử đứng sau. Khi chèn một bệnh nhân mới, vòng lặp đi qua các phần tử đang có thứ tự cao hơn hoặc bằng. Với hai người cùng mức, người cũ có STT nhỏ hơn và đứng trước, vì thế người mới được đặt sau. Trường hợp chèn vào đầu được xử lý riêng nếu danh sách rỗng hoặc đầu cũ có thứ tự thấp hơn. Lấy đầu chỉ gỡ nút đầu, nên phần còn lại vẫn có thứ tự. Ca P02 dùng các mức 2, 5, 3, 5 để xác nhận kết quả 5 trước 3 trước 2 và hai mức 5 vẫn FIFO.

Deque lưu lịch sử theo thứ tự từ cũ đến mới. Sau mỗi lượt gọi, hệ thống thêm cuối. Nếu trước khi thêm đã có 100 phần tử, hệ thống lấy đầu để bỏ đúng lượt cũ nhất. Bất biến con trỏ của danh sách đôi là dau.truoc rỗng, cuoi.sau rỗng và dem bằng số nút đi được từ dau tới cuoi. Khi xóa nút duy nhất, cả dau và cuoi đều phải trở về rỗng. D04 kiểm tra chính trường hợp này trước khi thêm lại, tránh để con trỏ cuối trỏ tới vùng nhớ đã giải phóng.

## Giữ trạng thái nhất quán khi gọi và chuyển khoa

Gọi số là phép biến đổi từ hàng chờ sang lịch sử. Sau một lượt gọi thành công, số người chờ giảm đúng một, người được gọi không còn trong Queue hoặc Priority Queue, và một bản ghi mới xuất hiện ở cuối Deque. Nếu gọi thất bại do hàng rỗng hoặc phòng không tồn tại, hàm trả false và không đổi các bộ đếm. H11-H15 kiểm tra thứ tự gọi theo phòng và toàn hệ thống. H22-H24 kiểm tra rằng hủy không tạo lịch sử, còn gọi thì có và lịch sử không vượt quá 100 lượt.

Chuyển cấp cứu có bất biến khác: số người chờ không đổi. Người bệnh được lấy khỏi hàng cũ, giữ mã và STT, cập nhật phòng đích cùng độ ưu tiên 999 rồi được chèn vào hàng ưu tiên đích.

Nếu bệnh nhân không tồn tại, đã ở khoa cấp cứu hoặc phòng đích không thể tạo, hàm trả false trước khi chuyển. Lệnh chuyển lặp không được âm thầm coi là thành công, vì người vận hành cần biết thao tác không thay đổi gì. H16 kiểm tra chuyển lần đầu thành công, lần hai và mã không tồn tại thất bại.

## Biên của ước tính thời gian chờ

Thời gian chờ tính riêng từng phòng. Ví dụ cùng phòng Nội có một người mức 5, hai người mức 3 theo STT và hai người thường: thứ tự phục vụ là mức 5, mức 3 đến trước, mức 3 đến sau, thường đến trước, thường đến sau. Mức 5 nhận 0 phút; người mức 3 thứ hai nhận 20 phút; người thường thứ nhất nhận 30 phút. Một người cấp cứu mới đến sau có thể được chen lên, nên ước tính trước đó phải được tính lại khi tra cứu. Hàm trả -1 khi không tìm thấy mã đang chờ, thay vì hiển thị 0 phút dễ gây hiểu nhầm.

Giả định 10 phút mỗi lượt làm kết quả dễ kiểm tra nhưng không phản ánh thời gian khám thực, thời gian chẩn đoán hay bác sĩ bận. Bản website hiện có còn có trạng thái phòng đang khám và có thể cộng thêm một lượt; bản console này chỉ tính trên hàng chờ. Hai mô hình không được trình bày là một hàm duy nhất hoặc một kết quả luôn bằng nhau. Khi thuyết trình cần nêu rõ đang demo console hay web.

## Giới hạn của việc lưu CSV và khả năng mở rộng

docCSV chia thành giai đoạn kiểm tra và giai đoạn áp dụng. Ở giai đoạn đầu, nó kiểm tra tiêu đề CAREQUEUE,1, STT, phòng, các bản ghi Q/H, mã trùng, STT trùng, sức chứa và độ ưu tiên hợp lệ. Chỉ sau khi toàn bộ file vượt qua kiểm tra, hàm datLai mới xóa trạng thái hiện tại để nạp bản mới. Nhờ vậy file có một dòng lỗi ở cuối không làm mất hàng chờ đang sử dụng. F08-F18 bao phủ file rỗng, sai tiêu đề, thiếu trường, trùng mã, ưu tiên ngoài khoảng, bản ghi lạ và dòng CRLF của Windows.

luuCSV ghi file tạm trước. Trên hệ thống cho phép đổi tên đè trực tiếp, việc thay file có tính nguyên tử ở mức thao tác đổi tên. Trên Windows, nhánh dự phòng xóa file đích rồi mới đổi tên, tạo một khoảng mất an toàn nếu thao tác sau thất bại. Vì vậy dự án chỉ dùng dữ liệu giả, không dùng để lưu hồ sơ thật, và nên sao lưu CSV trước khi thao tác với dữ liệu cần giữ. Nếu phát triển thành ứng dụng nhiều người dùng, cần chuyển sang cơ sở dữ liệu có giao dịch và kiểm soát ghi đồng thời.

Độ phức tạp khi gọi tự động là O(P) để so ứng viên đầu của tối đa 10 phòng. Tuy nhiên tìm một bệnh nhân theo mã duyệt các phòng và các hàng, trong khi xemTaiViTri(i) trên danh sách liên kết lại đi từ đầu. Nếu gọi lặp theo chỉ số, chi phí của một hàng ưu tiên có thể lên $O(U^2)$. Tương tự, liệt kê lịch sử bằng xemTaiViTri nhiều lần có thể là $O(H^2)$. Với U và H không quá

100, giới hạn này chấp nhận được cho bài tập. Nếu mở rộng cho bệnh viện thật, nên thêm bộ chỉ mục mã bệnh nhân và cách duyệt một lần qua danh sách.

# Kết luận và hướng phát triển

Đề tài đã thể hiện ba cấu trúc dữ liệu trong những vai trò khác nhau: Queue vòng bảo đảm FIFO cho khám thường, Priority Queue phục vụ ca khẩn theo mức ưu tiên và thứ tự đến, Deque giữ lịch sử với thao tác hai đầu. Toán tử so sánh trên BenhNhan là quy tắc thống nhất cho điều phối. Chương trình C++ console hỗ trợ tiếp nhận, phân luồng, gọi, chuyển cấp cứu, ước tính chờ, lưu/nạp và có bộ kiểm thử tự động.

Hướng phát triển hợp lý sau khi nghiệm thu bài tập lớn là tách lớp nghiệp vụ khỏi luồng nhập console để xây dựng giao diện web; thay CSV bằng cơ sở dữ liệu có giao dịch và khóa đồng thời; ghi thời gian khám thực tế để ước tính chờ theo từng phòng; phân quyền tài khoản vận hành; và lưu nhật ký thao tác. Những phần này không nằm trong phạm vi bản C++ console hiện tại.

## Bảng tổng hợp kiểm thử và độ phức tạp

| Cấu trúc | Thêm | Lấy đầu | Xóa theo mã |
|---|---|---|---|
| Queue mảng vòng | O(1) | O(1) | O(n) |
| Priority Queue DSLK | O(n) | O(1) | O(n) |
| Deque DSLK đôi | O(1) ở hai đầu | O(1) | Không dùng |

| Nhóm kiểm thử | Số ca | Nội dung |
|---|---|---|
| Queue | 7 | Rỗng, FIFO, đầy, quay vòng, xóa giữa |
| Priority Queue | 6 | Mức ưu tiên, FIFO cùng mức, xóa theo mã |
| Deque | 4 | Thêm/lấy hai đầu, rỗng rồi thêm lại |
| Nghiệp vụ | 28 | Tiếp nhận, phòng, gọi, cấp cứu, lịch sử, thời gian |
| CSV | 20 | Lưu/nạp, file lỗi, CRLF, giữ trạng thái |
| Tổng | 65 | 65/65 ca đạt trên bản console đóng gói |

# Tài liệu tham khảo và minh chứng

[1] Nhóm 6, mã nguồn CareQueue, https://github.com/tiettrungtinvn-create/NH-M-N-Y-T-Y (truy cập 08/11/2026).

[2] Nhóm 6, bảng nhiệm vụ Jira NNT, https://nguyentienthanh11207.atlassian.net/jira/software/projects/NNT/list (truy cập 08/11/2026).

[3] Cormen, Leiserson, Rivest, Stein, Introduction to Algorithms, ấn bản 4, MIT Press, 2022.

[4] ISO/IEC 14882:2017, Programming Languages - C++.

[5] Slide học phần Cấu trúc dữ liệu và Giải thuật do giảng viên Trần Thị Dung cung cấp. Nhóm cần bổ sung tên bài và trang tham chiếu nếu trích dẫn cụ thể.

# Phụ lục A. Mã nguồn C++ hiện hành

Các listing sau lấy từ thư mục console/ trên GitHub, đúng với bản kiểm thử 65/65. Đây là mã console độc lập, không phải mã web_server.cpp. Số dòng bên trái tương ứng từng tệp nguồn.

## include/BenhNhan.h

```
1  #ifndef BENHNHAN_H
2  #define BENHNHAN_H
3
4  #include <iostream>
5  #include <string>
6  using namespace std;
7
8  const int MAX_HANG_DOI = 100;
9  const int SO_PHONG_TOI_DA = 10;
10 const int PHUT_MOI_LUOT = 10;
11
12 struct BenhNhan {
13  int maBenhNhan;
14  string ten;
15  string phongKham;
16  int doUuTien;
17  int soThuTu;
18
19  BenhNhan() : maBenhNhan(0), ten(""), phongKham(""), doUuTien(0), soThuTu(0) {}
20
21  bool operator<(const BenhNhan& o) const {
22  if (doUuTien != o.doUuTien) return doUuTien < o.doUuTien;
23  return soThuTu > o.soThuTu;
24  }
25  bool operator>(const BenhNhan& o) const { return o < *this; }
26  bool operator==(const BenhNhan& o) const { return maBenhNhan == o.maBenhNhan; }
27 };
28
29 inline ostream& operator<<(ostream& os, const BenhNhan& bn) {
30  os << "STT " << bn.soThuTu << " | Ma " << bn.maBenhNhan << " | " << bn.ten
31     << " | " << bn.phongKham << " | uu tien " << bn.doUuTien;
32  return os;
33 }
34
35 #endif
```

## include/HangDoi.h

```
1  #ifndef HANGDOI_H
2  #define HANGDOI_H
3
4  #include "BenhNhan.h"
5
6  class HangDoi {
7  private:
8  BenhNhan a[MAX_HANG_DOI];
```

```
 9   int dau;
10   int dem;
11
12  public:
13   HangDoi() : dau(0), dem(0) {}
14
15   int kichThuoc() const { return dem; }
16   bool laRong() const { return dem == 0; }
17   bool laDay() const { return dem == MAX_HANG_DOI; }
18
19   bool them(const BenhNhan& bn) {
20   if (laDay()) return false;
21   a[(dau + dem) % MAX_HANG_DOI] = bn;
22   dem++;
23   return true;
24   }
25
26   bool lay(BenhNhan& bn) {
27   if (laRong()) return false;
28   bn = a[dau];
29   dau = (dau + 1) % MAX_HANG_DOI;
30   dem--;
31   return true;
32   }
33
34   bool xemDau(BenhNhan& bn) const {
35   if (laRong()) return false;
36   bn = a[dau];
37   return true;
38   }
39
40   bool xemTaiViTri(int i, BenhNhan& bn) const {
41   if (i < 0 || i >= dem) return false;
42   bn = a[(dau + i) % MAX_HANG_DOI];
43   return true;
44   }
45
46   bool layTheoMa(int ma, BenhNhan& bn) {
47   for (int i = 0; i < dem; i++) {
48   if (a[(dau + i) % MAX_HANG_DOI].maBenhNhan == ma) {
49   bn = a[(dau + i) % MAX_HANG_DOI];
50   for (int k = i; k < dem - 1; k++)
51   a[(dau + k) % MAX_HANG_DOI] = a[(dau + k + 1) % MAX_HANG_DOI];
52   dem--;
53   return true;
54   }
55   }
56   return false;
57   }
58  };
59
60  #endif
```

## include/HangUuTien.h

```
#ifndef HANGUUTIEN_H
#define HANGUUTIEN_H

#include "BenhNhan.h"

class HangUuTien {
private:
 struct Nut {
 BenhNhan du;
 Nut* tiep;
 Nut(const BenhNhan& x) : du(x), tiep(NULL) {}
 };
 Nut* dau;
 int dem;

public:
 HangUuTien() : dau(NULL), dem(0) {}
 ~HangUuTien() {
 while (dau != NULL) {
 Nut* t = dau;
 dau = dau->tiep;
 delete t;
 }
 }
 HangUuTien(const HangUuTien&) = delete;
 HangUuTien& operator=(const HangUuTien&) = delete;

 int kichThuoc() const { return dem; }
 bool laRong() const { return dem == 0; }

 bool them(const BenhNhan& bn) {
 if (dem == MAX_HANG_DOI) return false;
 Nut* p = new Nut(bn);
 if (dau == NULL || dau->du < bn) {
 p->tiep = dau;
 dau = p;
 } else {
 Nut* i = dau;
 while (i->tiep != NULL && !(i->tiep->du < bn)) i = i->tiep;
 p->tiep = i->tiep;
 i->tiep = p;
 }
 dem++;
 return true;
 }

 bool lay(BenhNhan& bn) {
 if (dau == NULL) return false;
 Nut* t = dau;
 bn = t->du;
 dau = dau->tiep;
 delete t;
 dem--;
 return true;
```

```
55   }
56
57   bool xemDau(BenhNhan& bn) const {
58   if (dau == NULL) return false;
59   bn = dau->du;
60   return true;
61   }
62
63   bool xemTaiViTri(int i, BenhNhan& bn) const {
64   if (i < 0 || i >= dem) return false;
65   Nut* p = dau;
66   for (int k = 0; k < i; k++) p = p->tiep;
67   bn = p->du;
68   return true;
69   }
70
71   bool layTheoMa(int ma, BenhNhan& bn) {
72   Nut* truoc = NULL;
73   for (Nut* i = dau; i != NULL; truoc = i, i = i->tiep) {
74   if (i->du.maBenhNhan == ma) {
75   bn = i->du;
76   if (truoc == NULL) dau = i->tiep;
77   else truoc->tiep = i->tiep;
78   delete i;
79   dem--;
80   return true;
81   }
82   }
83   return false;
84   }
85  };
86
87  #endif
```

## include/HangDoiHaiDau.h

```
1   #ifndef HANGDOIHAIDAU_H
2   #define HANGDOIHAIDAU_H
3
4   #include "BenhNhan.h"
5
6   class HangDoiHaiDau {
7   private:
8    struct Nut {
9    BenhNhan du;
10   Nut* truoc;
11   Nut* sau;
12   Nut(const BenhNhan& x) : du(x), truoc(NULL), sau(NULL) {}
13   };
14   Nut* dau;
15   Nut* cuoi;
16   int dem;
17
18  public:
19   HangDoiHaiDau() : dau(NULL), cuoi(NULL), dem(0) {}
```

```
20  ~HangDoiHaiDau() {
21  while (dau != NULL) {
22  Nut* t = dau;
23  dau = dau->sau;
24  delete t;
25  }
26  }
27  HangDoiHaiDau(const HangDoiHaiDau&) = delete;
28  HangDoiHaiDau& operator=(const HangDoiHaiDau&) = delete;
29
30  int kichThuoc() const { return dem; }
31  bool laRong() const { return dem == 0; }
32
33  bool themDau(const BenhNhan& bn) {
34  Nut* p = new Nut(bn);
35  if (dem == 0) {
36  dau = cuoi = p;
37  } else {
38  p->sau = dau;
39  dau->truoc = p;
40  dau = p;
41  }
42  dem++;
43  return true;
44  }
45
46  bool themCuoi(const BenhNhan& bn) {
47  Nut* p = new Nut(bn);
48  if (dem == 0) {
49  dau = cuoi = p;
50  } else {
51  p->truoc = cuoi;
52  cuoi->sau = p;
53  cuoi = p;
54  }
55  dem++;
56  return true;
57  }
58
59  bool layDau(BenhNhan& bn) {
60  if (dem == 0) return false;
61  Nut* t = dau;
62  bn = t->du;
63  dau = dau->sau;
64  if (dau == NULL) cuoi = NULL;
65  else dau->truoc = NULL;
66  delete t;
67  dem--;
68  return true;
69  }
70
71  bool layCuoi(BenhNhan& bn) {
72  if (dem == 0) return false;
73  Nut* t = cuoi;
74  bn = t->du;
75  cuoi = cuoi->truoc;
```

```
76   if (cuoi == NULL) dau = NULL;
77   else cuoi->sau = NULL;
78   delete t;
79   dem--;
80   return true;
81   }
82
83   bool xemTaiViTri(int i, BenhNhan& bn) const {
84   if (i < 0 || i >= dem) return false;
85   Nut* p = dau;
86   for (int k = 0; k < i; k++) p = p->sau;
87   bn = p->du;
88   return true;
89   }
90  };
91
92  #endif
```

## include/HeThongBenhVien.h

```
 1  #ifndef HETHONGBENHVIEN_H
 2  #define HETHONGBENHVIEN_H
 3
 4  #include "BenhNhan.h"
 5  #include "HangDoi.h"
 6  #include "HangDoiHaiDau.h"
 7  #include "HangUuTien.h"
 8
 9  const int DO_UU_TIEN_CAP_CUU = 999;   // muc uu tien cua ca chuyen sang Khoa Cap Cuu
10  const int DO_DAI_TEN_TOI_DA = 50;
11
12  struct Phong {
13   string ten;
14   HangUuTien hangCapCuu;   // benh nhan uu tien (doUuTien >= 2) - Priority Queue
15   HangDoi hangThuong;      // benh nhan thuong (doUuTien == 1) - Queue vong, FIFO
16  };
17
18  class HeThongBenhVien {
19  private:
20   Phong phong[SO_PHONG_TOI_DA];
21   int soPhong;
22   int sucChua;
23   int soLuong;
24   int sttTuDong;
25   HangDoiHaiDau lichSuGoi;     // Deque: lich su goi so
26   string loiGanNhat;           // ly do that bai cua thao tac gan nhat
27
28   int timPhong(string tenPhong) const;
29   int taoPhong(string tenPhong);
30   bool timBenhNhan(int ma, BenhNhan &bn, int &viTriPhong, bool &capCuu) const;
31   static bool tenHopLe(const string &s);
32   bool datLoi(const string &msg) { loiGanNhat = msg; return false; }
33
34  public:
35   HeThongBenhVien(int sc = MAX_HANG_DOI);
```

```
36   HeThongBenhVien(const HeThongBenhVien&) = delete;
37   HeThongBenhVien& operator=(const HeThongBenhVien&) = delete;
38
39   bool tiepNhanVaPhanLuong(int ma, string ten, string tenPhong, int doUuTien);
40   bool chuyenKhoaCapCuuUuTien(int maBN, string tenKhoa = "Khoa Cap Cuu");
41   bool goiSoTuDong(string tenPhong = "");
42   bool layBenhNhanTiepTheo(string tenPhong, BenhNhan &bn);
43   bool goiTheoMa(int maBN, BenhNhan &bn);
44   bool huyBenhNhan(int maBN);
45   int thoiGianChoDuKien(int maBN) const;
46   int lietKeThuTuCho(BenhNhan danhSach[], int sucChuaMang) const;
47   void luuLichSuGoi(const BenhNhan &bn);
48   int lietKeLichSuGoi(BenhNhan danhSach[], int sucChuaMang) const;
49   void uocTinhThoiGianCho(int maBN);
50   void hienThiBangDienTu();
51   void thongKePhongKham(string tenPhong);
52   void hienThiLichSuGoi();
53   bool laRong();
54   int kichThuoc();
55   int soPhongHienCo() const { return soPhong; }
56   int sucChuaToiDa() const { return sucChua; }
57   const string& loiCuoi() const { return loiGanNhat; }
58
59   // --- Luu tru ben vung (CSV) ---
60   // Ghi/doc toan bo trang thai (hang doi, STT, lich su). Loi -> nem std::runtime_error.
61   void datLai();                                    // xoa sach he thong
62   void luuCSV(const string &duongDan) const;       // ghi atomic (file tam + rename)
63   void docCSV(const string &duongDan);             // kiem tra toan bo roi moi nap (khong nap
     do dang)
64 };
65
66 #endif
```

## src/HeThong_TiepNha.cpp

```
1  #include "HeThongBenhVien.h"
2
3  HeThongBenhVien::HeThongBenhVien(int sc) {
4      soPhong = 0;
5      if (sc > 0 && sc <= MAX_HANG_DOI) {
6          sucChua = sc;
7      } else {
8          sucChua = MAX_HANG_DOI;
9      }
10     soLuong = 0;
11     sttTuDong = 0;
12 }
13
14 // Ten hop le: khong rong, khong chi gom khoang trang, <= 50 ky tu,
15 // khong chua dau phay / xuong dong (de luu CSV an toan).
16 bool HeThongBenhVien::tenHopLe(const string &s) {
17     if (s.empty() || s.size() > (size_t)DO_DAI_TEN_TOI_DA) return false;
18     bool coChu = false;
19     for (size_t i = 0; i < s.size(); i++) {
20         char c = s[i];
```

```
21          if (c == ',' || c == '\n' || c == '\r') return false;
22          if (c != ' ' && c != '\t') coChu = true;
23      }
24      return coChu;
25  }
26
27  int HeThongBenhVien::timPhong(string tenPhong) const {
28      for (int i = 0; i < soPhong; i++)
29          if (phong[i].ten == tenPhong) return i;
30      return -1;
31  }
32
33  int HeThongBenhVien::taoPhong(string tenPhong) {
34      int viTri = timPhong(tenPhong);
35      if (viTri >= 0) return viTri;
36      if (!tenHopLe(tenPhong) || soPhong == SO_PHONG_TOI_DA) return -1;
37      phong[soPhong].ten = tenPhong;
38      soPhong++;
39      return soPhong - 1;
40  }
41
42  bool HeThongBenhVien::timBenhNhan(int ma, BenhNhan &bn, int &viTriPhong, bool &capCuu) const
     {
43      for (int i = 0; i < soPhong; i++) {
44          for (int j = 0; j < phong[i].hangCapCuu.kichThuoc(); j++) {
45              phong[i].hangCapCuu.xemTaiViTri(j, bn);
46              if (bn.maBenhNhan == ma) {
47                  viTriPhong = i;
48                  capCuu = true;
49                  return true;
50              }
51          }
52          for (int j = 0; j < phong[i].hangThuong.kichThuoc(); j++) {
53              phong[i].hangThuong.xemTaiViTri(j, bn);
54              if (bn.maBenhNhan == ma) {
55                  viTriPhong = i;
56                  capCuu = false;
57                  return true;
58              }
59          }
60      }
61      return false;
62  }
63
64  bool HeThongBenhVien::tiepNhanVaPhanLuong(int ma, string ten, string tenPhong, int doUuTien)
     {
65      loiGanNhat.clear();
66      if (ma <= 0) return datLoi("Ma benh nhan phai la so duong.");
67      if (!tenHopLe(ten)) return datLoi("Ho ten khong hop le (rong, qua 50 ky tu hoac chua dau
     phay).");
68      if (!tenHopLe(tenPhong)) return datLoi("Ten phong khong hop le (rong, qua 50 ky tu hoac
    chua dau phay).");
69      if (doUuTien < 1 || doUuTien > 5) return datLoi("Do uu tien phai nam trong khoang 1..5."
    );
70      if (soLuong == sucChua) return datLoi("He thong da day (" + to_string(sucChua) + " benh
    nhan).");
```

```
    BenhNhan daCo;
    int viTriCu;
    bool capCuuCu;
    if (timBenhNhan(ma, daCo, viTriCu, capCuuCu)) return datLoi("Ma benh nhan " + to_string(
ma) + " da ton tai.");
    int viTri = taoPhong(tenPhong);
    if (viTri < 0) return datLoi("Da dat so phong toi da (" + to_string(SO_PHONG_TOI_DA) + "
).");

    BenhNhan bn;
    bn.maBenhNhan = ma;
    bn.ten = ten;
    bn.phongKham = tenPhong;
    bn.doUuTien = doUuTien;
    bn.soThuTu = sttTuDong + 1;
    bool daThem;
    if (doUuTien == 1) {
        daThem = phong[viTri].hangThuong.them(bn);
    } else {
        daThem = phong[viTri].hangCapCuu.them(bn);
    }
    if (!daThem) return datLoi("Hang doi cua phong " + tenPhong + " da day.");
    sttTuDong++;
    soLuong++;
    return true;
}
```

## src/HeThong_GoiSo.cpp

```
#include "HeThongBenhVien.h"

bool HeThongBenhVien::layBenhNhanTiepTheo(string tenPhong, BenhNhan &bn) {
    loiGanNhat.clear();
    if (laRong()) return datLoi("Khong co benh nhan nao dang cho.");
    int viTri = -1;
    bool capCuu = false;
    BenhNhan totNhat, ungVien;

    if (!tenPhong.empty()) {
        viTri = timPhong(tenPhong);
        if (viTri < 0) return datLoi("Phong \"" + tenPhong + "\" khong ton tai.");
        capCuu = !phong[viTri].hangCapCuu.laRong();
        if (!capCuu && phong[viTri].hangThuong.laRong())
            return datLoi("Phong \"" + tenPhong + "\" khong co benh nhan dang cho.");
    } else {
        for (int i = 0; i < soPhong; i++) {
            if (phong[i].hangCapCuu.xemDau(ungVien) &&
                (viTri < 0 || !capCuu || totNhat < ungVien)) {
                viTri = i;
                capCuu = true;
                totNhat = ungVien;
            }
        }
        if (viTri < 0) {
            for (int i = 0; i < soPhong; i++) {
```

```
                if (phong[i].hangThuong.xemDau(ungVien) &&
                    (viTri < 0 || ungVien.soThuTu < totNhat.soThuTu)) {
                    viTri = i;
                    totNhat = ungVien;
                }
            }
        }
    }

    if (capCuu) phong[viTri].hangCapCuu.lay(bn);
    else phong[viTri].hangThuong.lay(bn);
    soLuong--;
    luuLichSuGoi(bn);
    return true;
}

void HeThongBenhVien::luuLichSuGoi(const BenhNhan &bn) {
    if (lichSuGoi.kichThuoc() == MAX_HANG_DOI) {
        BenhNhan cu;
        lichSuGoi.layDau(cu);
    }
    lichSuGoi.themCuoi(bn);
}

bool HeThongBenhVien::goiSoTuDong(string tenPhong) {
    BenhNhan bn;
    if (!layBenhNhanTiepTheo(tenPhong, bn)) return false;
    cout << "[LOA THONG BAO] Moi BN STT " << bn.soThuTu << " - " << bn.ten
         << " vao " << bn.phongKham << "!\n";
    return true;
}

bool HeThongBenhVien::goiTheoMa(int maBN, BenhNhan &bn) {
    loiGanNhat.clear();
    int viTri;
    bool capCuu;
    if (!timBenhNhan(maBN, bn, viTri, capCuu))
        return datLoi("Khong tim thay benh nhan " + to_string(maBN) + ".");
    bool daLay;
    if (capCuu) daLay = phong[viTri].hangCapCuu.layTheoMa(maBN, bn);
    else daLay = phong[viTri].hangThuong.layTheoMa(maBN, bn);
    if (!daLay) return false;
    soLuong--;
    luuLichSuGoi(bn);
    return true;
}

bool HeThongBenhVien::huyBenhNhan(int maBN) {
    loiGanNhat.clear();
    BenhNhan bn;
    int viTri;
    bool capCuu;
    if (!timBenhNhan(maBN, bn, viTri, capCuu))
        return datLoi("Khong tim thay benh nhan " + to_string(maBN) + ".");
    bool daLay;
    if (capCuu) daLay = phong[viTri].hangCapCuu.layTheoMa(maBN, bn);
```

```
    else daLay = phong[viTri].hangThuong.layTheoMa(maBN, bn);
    if (daLay) soLuong--;
    return daLay;
}

void HeThongBenhVien::hienThiLichSuGoi() {
    cout << "--- LICH SU GOI (cu -> moi, toi da " << MAX_HANG_DOI << ") ---\n";
    if (lichSuGoi.laRong()) {
        cout << "  (chua goi ai)\n";
        return;
    }
    BenhNhan bn;
    for (int i = 0; i < lichSuGoi.kichThuoc(); i++) {
        lichSuGoi.xemTaiViTri(i, bn);
        cout << "  STT " << bn.soThuTu << " | Ma " << bn.maBenhNhan << " | " << bn.ten
             << " | " << bn.phongKham << "\n";
    }
}
int HeThongBenhVien::lietKeLichSuGoi(BenhNhan danhSach[], int sucChuaMang) const {
    if (danhSach == NULL || sucChuaMang <= 0) return 0;
    int n = 0;
    for (int i = 0; i < lichSuGoi.kichThuoc() && n < sucChuaMang; i++) {
        lichSuGoi.xemTaiViTri(i, danhSach[n]);
        n++;
    }
    return n;
}
```

## src/HeThong_ChuyenKhoa.cpp

```
#include "HeThongBenhVien.h"

bool HeThongBenhVien::chuyenKhoaCapCuuUuTien(int maBN, string tenKhoa) {
    loiGanNhat.clear();
    BenhNhan bn;
    int viTriCu;
    bool capCuuCu;
    if (!timBenhNhan(maBN, bn, viTriCu, capCuuCu))
        return datLoi("Khong tim thay benh nhan " + to_string(maBN) + " trong hang cho.");
    if (bn.phongKham == tenKhoa && bn.doUuTien == DO_UU_TIEN_CAP_CUU)
        return datLoi("Benh nhan da o " + tenKhoa + " voi uu tien cao nhat.");
    int viTriMoi = taoPhong(tenKhoa);
    if (viTriMoi < 0) return datLoi("Khong tao duoc khoa dich (ten sai hoac da du so phong).
");
    if (phong[viTriMoi].hangCapCuu.kichThuoc() == MAX_HANG_DOI &&
        !(viTriCu == viTriMoi && capCuuCu))
        return datLoi("Hang cap cuu cua " + tenKhoa + " da day.");

    if (capCuuCu) phong[viTriCu].hangCapCuu.layTheoMa(maBN, bn);
    else phong[viTriCu].hangThuong.layTheoMa(maBN, bn);
    bn.phongKham = tenKhoa;
    bn.doUuTien = DO_UU_TIEN_CAP_CUU;
    phong[viTriMoi].hangCapCuu.them(bn);
    cout << "[THONG BAO KHAN] Da chuyen BN " << maBN << " sang " << tenKhoa << "!\n";
    return true;
```

```
}
```

## src/HeThong_ThongKe.cpp

```
#include <iomanip>
#include "HeThongBenhVien.h"
int HeThongBenhVien::lietKeThuTuCho(BenhNhan danhSach[], int sucChuaMang) const {
    if (danhSach == NULL || sucChuaMang <= 0) return 0;
    BenhNhan tatCa[MAX_HANG_DOI];
    int dem = 0;
    for (int i = 0; i < soPhong; i++) {
        for (int j = 0; j < phong[i].hangCapCuu.kichThuoc(); j++)
            phong[i].hangCapCuu.xemTaiViTri(j, tatCa[dem++]);
        for (int j = 0; j < phong[i].hangThuong.kichThuoc(); j++)
            phong[i].hangThuong.xemTaiViTri(j, tatCa[dem++]);
    }
    for (int i = 0; i < dem; i++) {
        int totNhat = i;
        for (int j = i + 1; j < dem; j++)
            if (tatCa[totNhat] < tatCa[j]) totNhat = j;
        BenhNhan tam = tatCa[i];
        tatCa[i] = tatCa[totNhat];
        tatCa[totNhat] = tam;
    }
    if (dem > sucChuaMang) dem = sucChuaMang;
    for (int i = 0; i < dem; i++) danhSach[i] = tatCa[i];
    return dem;
}
int HeThongBenhVien::thoiGianChoDuKien(int maBN) const {
    BenhNhan bn, dangXet;
    int viTri;
    bool capCuu;
    if (!timBenhNhan(maBN, bn, viTri, capCuu)) return -1;
    int truoc;
    if (capCuu) {
        truoc = 0;
    } else {
        truoc = phong[viTri].hangCapCuu.kichThuoc();
    }
    if (capCuu) {
        for (int i = 0; i < phong[viTri].hangCapCuu.kichThuoc(); i++) {
            phong[viTri].hangCapCuu.xemTaiViTri(i, dangXet);
            if (dangXet.maBenhNhan == maBN) break;
            truoc++;
        }
    } else {
        for (int i = 0; i < phong[viTri].hangThuong.kichThuoc(); i++) {
            phong[viTri].hangThuong.xemTaiViTri(i, dangXet);
            if (dangXet.maBenhNhan == maBN) break;
            truoc++;
        }
    }
    return truoc * PHUT_MOI_LUOT;
}

```

```
52  void HeThongBenhVien::uocTinhThoiGianCho(int maBN) {
53      int phut = thoiGianChoDuKien(maBN);
54      if (phut < 0) cout << "[LOI] Khong tim thay BN " << maBN << " trong hang cho.\n";
55      else cout << "[TRA CUU] BN " << maBN << " can cho khoang " << phut << " phut.\n";
56  }
57
58  void HeThongBenhVien::hienThiBangDienTu() {
59      if (laRong()) {
60          cout << "--- BANG DIEN TU: TRONG ---\n";
61          return;
62      }
63      cout << "--- BANG DIEN TU (" << soLuong << " benh nhan dang cho) ---\n";
64      BenhNhan bn;
65      for (int i = 0; i < soPhong; i++) {
66          if (phong[i].hangCapCuu.laRong() && phong[i].hangThuong.laRong()) continue;
67          cout << "Phong: " << phong[i].ten << "\n";
68          cout << "  " << left << setw(10) << "Loai" << setw(6) << "STT" << setw(9) << "Ma"
69               << setw(26) << "Ho ten" << "Cho(phut)\n";
70          int viTri = 0;
71          for (int j = 0; j < phong[i].hangCapCuu.kichThuoc(); j++, viTri++) {
72              phong[i].hangCapCuu.xemTaiViTri(j, bn);
73              string loai = "UT " + to_string(bn.doUuTien);
74              cout << "  " << left << setw(10) << loai << setw(6) << bn.soThuTu << setw(9) <<
    bn.maBenhNhan
75                   << setw(26) << bn.ten << viTri * PHUT_MOI_LUOT << "\n";
76          }
77          for (int j = 0; j < phong[i].hangThuong.kichThuoc(); j++, viTri++) {
78              phong[i].hangThuong.xemTaiViTri(j, bn);
79              cout << "  " << left << setw(10) << "THUONG" << setw(6) << bn.soThuTu << setw(9)
     << bn.maBenhNhan
80                   << setw(26) << bn.ten << viTri * PHUT_MOI_LUOT << "\n";
81          }
82      }
83  }
84  void HeThongBenhVien::thongKePhongKham(string tenPhong) {
85      int viTri = timPhong(tenPhong);
86      int dem;
87      if (viTri < 0) {
88          dem = 0;
89      } else {
90          dem = phong[viTri].hangCapCuu.kichThuoc() + phong[viTri].hangThuong.kichThuoc();
91      }
92      if (viTri < 0) {
93          cout << "[THONG KE] Phong \"" << tenPhong << "\" khong ton tai (0 BN).\n";
94          return;
95      }
96      cout << "[THONG KE] Phong " << tenPhong << ": " << dem << " BN xep hang ("
97           << phong[viTri].hangCapCuu.kichThuoc() << " uu tien, "
98           << phong[viTri].hangThuong.kichThuoc() << " thuong).\n";
99  }
100
101 bool HeThongBenhVien::laRong() { return soLuong == 0; }
102 int HeThongBenhVien::kichThuoc() { return soLuong; }
103
```

## src/HeThong_LuuTru.cpp

```cpp
// Luu / doc trang thai he thong ra file CSV.
//
// Dinh dang (moi dong mot ban ghi, phan tach bang dau phay):
//   CAREQUEUE,1                     <- dau file + phien ban
//   STT,<so thu tu da cap>
//   R,<ten phong>                   <- phong (giu nguyen thu tu)
//   Q,<ma>,<ten>,<phong>,<uu tien>,<stt>   <- benh nhan dang cho (theo thu tu hang)
//   H,<ma>,<ten>,<phong>,<uu tien>,<stt>   <- lich su goi (cu -> moi)
#include <cstdio>
#include <fstream>
#include <set>
#include <sstream>
#include <stdexcept>
#include <vector>
#include <filesystem>
#include "HeThongBenhVien.h"

namespace {

struct BanGhi {
    char loai;       // 'Q' hoac 'H'
    BenhNhan bn;
    int dong;        // so dong trong file (de bao loi)
};

[[noreturn]] void loiDong(int dong, const string &msg) {
    throw runtime_error("Dong " + to_string(dong) + ": " + msg);
}

vector<string> tach(const string &line) {
    vector<string> out;
    string cur;
    for (size_t i = 0; i < line.size(); i++) {
        if (line[i] == ',') { out.push_back(cur); cur.clear(); }
        else cur += line[i];
    }
    out.push_back(cur);
    return out;
}

int docSo(const string &s, int dong, const char *tenTruong) {
    if (s.empty()) loiDong(dong, string("thieu gia tri ") + tenTruong);
    size_t pos = 0;
    long long v;
    try {
        v = stoll(s, &pos);
    } catch (const exception &) {
        loiDong(dong, string(tenTruong) + " khong phai so: \"" + s + "\"");
    }
    if (pos != s.size()) loiDong(dong, string(tenTruong) + " khong phai so: \"" + s + "\"");
    if (v < -2000000000LL || v > 2000000000LL) loiDong(dong, string(tenTruong) + " ngoai pha
m vi");
    return (int)v;
}
```

```
54
55  }  // namespace
56
57  void HeThongBenhVien::datLai() {
58      BenhNhan tam;
59      for (int i = 0; i < SO_PHONG_TOI_DA; i++) {
60          while (phong[i].hangCapCuu.lay(tam)) {}
61          while (phong[i].hangThuong.lay(tam)) {}
62          phong[i].ten.clear();
63      }
64      while (lichSuGoi.layDau(tam)) {}
65      soPhong = 0;
66      soLuong = 0;
67      sttTuDong = 0;
68      loiGanNhat.clear();
69  }
70
71  void HeThongBenhVien::luuCSV(const string &duongDan) const {
72      if (duongDan.empty()) throw runtime_error("Duong dan file rong.");
73      filesystem::path dich(duongDan);
74      string tam = duongDan + ".tmp";
75      try {
76          if (dich.has_parent_path()) filesystem::create_directories(dich.parent_path());
77      } catch (const filesystem::filesystem_error &e) {
78          throw runtime_error(string("Khong tao duoc thu muc: ") + e.what());
79      }
80      {
81          ofstream f(tam.c_str(), ios::out | ios::trunc);
82          if (!f) throw runtime_error("Khong mo duoc file de ghi: " + tam);
83          f << "CAREQUEUE,1\n";
84          f << "STT," << sttTuDong << "\n";
85          for (int i = 0; i < soPhong; i++) f << "R," << phong[i].ten << "\n";
86          BenhNhan bn;
87          for (int i = 0; i < soPhong; i++) {
88              for (int j = 0; j < phong[i].hangCapCuu.kichThuoc(); j++) {
89                  phong[i].hangCapCuu.xemTaiViTri(j, bn);
90                  f << "Q," << bn.maBenhNhan << ',' << bn.ten << ',' << bn.phongKham << ','
91                    << bn.doUuTien << ',' << bn.soThuTu << "\n";
92              }
93              for (int j = 0; j < phong[i].hangThuong.kichThuoc(); j++) {
94                  phong[i].hangThuong.xemTaiViTri(j, bn);
95                  f << "Q," << bn.maBenhNhan << ',' << bn.ten << ',' << bn.phongKham << ','
96                    << bn.doUuTien << ',' << bn.soThuTu << "\n";
97              }
98          }
99          for (int j = 0; j < lichSuGoi.kichThuoc(); j++) {
100             lichSuGoi.xemTaiViTri(j, bn);
101             f << "H," << bn.maBenhNhan << ',' << bn.ten << ',' << bn.phongKham << ','
102               << bn.doUuTien << ',' << bn.soThuTu << "\n";
103         }
104         f.flush();
105         if (!f) {
106             f.close();
107             remove(tam.c_str());
108             throw runtime_error("Ghi file that bai (day dia / mat quyen?): " + tam);
109         }
```

```
110     }
111     error_code ec;
112     filesystem::rename(tam, dich, ec);
113     if (ec) {   // Windows: rename khong ghi de duoc -> xoa roi doi ten lai
114         filesystem::remove(dich, ec);
115         error_code ec2;
116         filesystem::rename(tam, dich, ec2);
117         if (ec2) throw runtime_error("Khong thay the duoc file dich: " + duongDan);
118     }
119 }
120 
121 void HeThongBenhVien::docCSV(const string &duongDan) {
122     ifstream f(duongDan.c_str());
123     if (!f) throw runtime_error("Khong mo duoc file de doc: " + duongDan);
124 
125     // ---- Giai doan 1: doc + kiem tra toan bo, CHUA dong vao he thong ----
126     string line;
127     int dong = 0;
128     if (!getline(f, line)) throw runtime_error("File rong.");
129     dong++;
130     if (!line.empty() && line[line.size() - 1] == '\r') line.erase(line.size() - 1);
131     if (line != "CAREQUEUE,1") throw runtime_error("Sai dinh dang file (thieu dong \"CAREQUE
    UE,1\").");
132 
133     int stt = -1;
134     vector<string> dsPhong;
135     vector<BanGhi> dsQ, dsH;
136     set<int> maDaCo, sttDaCo;
137 
138     while (getline(f, line)) {
139         dong++;
140         if (!line.empty() && line[line.size() - 1] == '\r') line.erase(line.size() - 1);
141         if (line.empty()) continue;
142         vector<string> t = tach(line);
143         const string &loai = t[0];
144         if (loai == "STT") {
145             if (t.size() != 2) loiDong(dong, "dong STT phai co 2 truong");
146             if (stt >= 0) loiDong(dong, "dong STT bi lap");
147             stt = docSo(t[1], dong, "STT");
148             if (stt < 0) loiDong(dong, "STT am");
149         } else if (loai == "R") {
150             if (t.size() != 2) loiDong(dong, "dong R phai co 2 truong");
151             if (!tenHopLe(t[1])) loiDong(dong, "ten phong khong hop le");
152             for (size_t i = 0; i < dsPhong.size(); i++)
153                 if (dsPhong[i] == t[1]) loiDong(dong, "phong bi lap: " + t[1]);
154             dsPhong.push_back(t[1]);
155         } else if (loai == "Q" || loai == "H") {
156             if (t.size() != 6) loiDong(dong, "dong " + loai + " phai co 6 truong");
157             BanGhi g;
158             g.loai = loai[0];
159             g.dong = dong;
160             g.bn.maBenhNhan = docSo(t[1], dong, "ma");
161             g.bn.ten = t[2];
162             g.bn.phongKham = t[3];
163             g.bn.doUuTien = docSo(t[4], dong, "uu tien");
164             g.bn.soThuTu = docSo(t[5], dong, "stt");
```

```
165             if (g.bn.maBenhNhan <= 0) loiDong(dong, "ma benh nhan phai duong");
166             if (!tenHopLe(g.bn.ten)) loiDong(dong, "ho ten khong hop le");
167             if (!tenHopLe(g.bn.phongKham)) loiDong(dong, "ten phong khong hop le");
168             if (!((g.bn.doUuTien >= 1 && g.bn.doUuTien <= 5) || g.bn.doUuTien == DO_UU_TIEN_
    CAP_CUU))
169                 loiDong(dong, "do uu tien ngoai 1..5 / 999");
170             if (g.bn.soThuTu <= 0) loiDong(dong, "STT phai duong");
171             (g.loai == 'Q' ? dsQ : dsH).push_back(g);
172         } else {
173             loiDong(dong, "loai ban ghi la: \"" + loai + "\"");
174         }
175     }
176     if (f.bad()) throw runtime_error("Loi khi doc file.");
177     if (stt < 0) throw runtime_error("File thieu dong STT.");
178 
179     // Rang buoc nghiep vu
180     vector<string> tatCaPhong = dsPhong;
181     auto themPhong = [&](const string &ten) {
182         for (size_t i = 0; i < tatCaPhong.size(); i++) if (tatCaPhong[i] == ten) return;
183         tatCaPhong.push_back(ten);
184     };
185     int demHang[SO_PHONG_TOI_DA + 1][2] = {};
186     for (size_t i = 0; i < dsQ.size(); i++) {
187         const BenhNhan &b = dsQ[i].bn;
188         if (!maDaCo.insert(b.maBenhNhan).second) loiDong(dsQ[i].dong, "ma benh nhan bi trung
    ");
189         if (!sttDaCo.insert(b.soThuTu).second) loiDong(dsQ[i].dong, "STT bi trung");
190         if (b.soThuTu > stt) loiDong(dsQ[i].dong, "STT lon hon STT da cap");
191         themPhong(b.phongKham);
192     }
193     for (size_t i = 0; i < dsH.size(); i++) {
194         if (dsH[i].bn.soThuTu > stt) loiDong(dsH[i].dong, "STT lon hon STT da cap");
195         themPhong(dsH[i].bn.phongKham);
196     }
197     if ((int)tatCaPhong.size() > SO_PHONG_TOI_DA) throw runtime_error("Vuot qua so phong toi
     da.");
198     if ((int)dsQ.size() > sucChua) throw runtime_error("So benh nhan trong file vuot suc chu
    a he thong.");
199     if ((int)dsH.size() > MAX_HANG_DOI) throw runtime_error("Lich su goi vuot gioi han.");
200     for (size_t i = 0; i < dsQ.size(); i++) {
201         size_t p = 0;
202         while (tatCaPhong[p] != dsQ[i].bn.phongKham) p++;
203         int k = (dsQ[i].bn.doUuTien == 1) ? 0 : 1;
204         if (++demHang[p][k] > MAX_HANG_DOI) loiDong(dsQ[i].dong, "hang doi cua phong bi qua
    day");
205     }
206 
207     // ---- Giai doan 2: da hop le -> nap vao he thong ----
208     datLai();
209     sttTuDong = stt;
210     for (size_t i = 0; i < tatCaPhong.size(); i++) taoPhong(tatCaPhong[i]);
211     for (size_t i = 0; i < dsQ.size(); i++) {
212         const BenhNhan &b = dsQ[i].bn;
213         int vt = timPhong(b.phongKham);
214         bool ok = (b.doUuTien == 1) ? phong[vt].hangThuong.them(b) : phong[vt].hangCapCuu.th
    em(b);
```

```
        if (!ok) throw logic_error("Loi noi bo khi nap du lieu.");
        soLuong++;
    }
    for (size_t i = 0; i < dsH.size(); i++) lichSuGoi.themCuoi(dsH[i].bn);
}
```

## src/main.cpp

```
// CareQueue UTC2 - He thong xep hang kham benh & cap cuu (Console)
// Cau truc du lieu: Queue vong (HangDoi), Priority Queue lien ket (HangUuTien),
//                   Deque lien ket doi (HangDoiHaiDau).
#include <cstdlib>
#include <cstdio>
#include <exception>
#include <filesystem>
#include <iomanip>
#include <limits>
#include <stdexcept>
#include "HeThongBenhVien.h"

namespace {

const char *FILE_MAC_DINH = "data/benhvien.csv";

// Nem khi het du lieu nhap (Ctrl+D / Ctrl+Z, hoac stdin dong) -> thoat sach se.
struct HetDuLieuNhap {};

void kq(const string &viec, bool ok, const HeThongBenhVien *ht = nullptr) {
 cout << "  -> " << viec << ": " << (ok ? "THANH CONG" : "THAT BAI");
 if (!ok && ht != nullptr && !ht->loiCuoi().empty()) cout << "  (" << ht->loiCuoi() << ")";
 cout << "\n";
}

string catKhoangTrang(const string &s) {
 size_t a = s.find_first_not_of(" \t\r\n");
 if (a == string::npos) return "";
 size_t b = s.find_last_not_of(" \t\r\n");
 return s.substr(a, b - a + 1);
}

string docDong(const string &loiNhac) {
 string s;
 cout << loiNhac;
 if (!getline(cin, s)) throw HetDuLieuNhap();
 return catKhoangTrang(s);
}

// Doc so nguyen trong [lo, hi]; nhap sai thi bao loi va hoi lai.
int docSo(const string &loiNhac, int lo, int hi) {
 while (true) {
 string s = docDong(loiNhac);
 size_t pos = 0;
 bool ok = !s.empty();
 long long v = 0;
 if (ok) {
```

```
 try { v = stoll(s, &pos); }
 catch (const exception &) { ok = false; }
 }
 if (ok && pos == s.size() && v >= lo && v <= hi) return (int)v;
 cout << "  ! Gia tri khong hop le - nhap mot so nguyen trong khoang " << lo << ".." << hi
<< ".\n";
 }
}

string docChuoiKhongRong(const string &loiNhac) {
 while (true) {
 string s = docDong(loiNhac);
 if (!s.empty()) return s;
 cout << "  ! Khong duoc de trong.\n";
 }
}

void luuTuDong(const HeThongBenhVien &ht, const string &file) {
 try {
 ht.luuCSV(file);
 } catch (const exception &e) {
 cout << "  [CANH BAO] Khong luu duoc du lieu: " << e.what() << "\n";
 }
}

// ------------------------------------------------------------------ demo
void chayDemo() {
 HeThongBenhVien ht(50);
 cout << "### KICH BAN DEMO ###\n\n-- 1. Tiep nhan va phan luong --\n";
 kq("BN 101 (thuong, Noi)",      ht.tiepNhanVaPhanLuong(101, "Nguyen Van A", "Noi", 1), &ht)
;
 kq("BN 102 (uu tien 3, Noi)",   ht.tiepNhanVaPhanLuong(102, "Tran Thi B",   "Noi", 3), &ht)
;
 kq("BN 103 (thuong, Ngoai)",    ht.tiepNhanVaPhanLuong(103, "Le Van C",     "Ngoai", 1), &h
t);
 kq("BN 104 (uu tien 5, Noi)",   ht.tiepNhanVaPhanLuong(104, "Pham Thi D",   "Noi", 5), &ht)
;
 kq("BN 105 (uu tien 2, Nhi)",   ht.tiepNhanVaPhanLuong(105, "Hoang Van E",  "Nhi", 2), &ht)
;
 kq("BN 106 (thuong, Noi)",      ht.tiepNhanVaPhanLuong(106, "Do Thi G",     "Noi", 1), &ht)
;

 cout << "\n-- 2. Cac truong hop loi --\n";
 kq("Trung ma 101",              ht.tiepNhanVaPhanLuong(101, "Trung Ma",     "Noi", 1), &ht)
;
 kq("Do uu tien 9 khong hop le", ht.tiepNhanVaPhanLuong(107, "Sai Muc",      "Noi", 9), &ht)
;
 kq("Ten rong",                  ht.tiepNhanVaPhanLuong(108, "",             "Noi", 1), &ht)
;
 kq("Ten co dau phay",           ht.tiepNhanVaPhanLuong(109, "A,B",          "Noi", 1), &ht)
;

 cout << "\n";
 ht.hienThiBangDienTu();

 cout << "\n-- 3. Uoc tinh thoi gian cho --\n";
```

```
 93   ht.uocTinhThoiGianCho(104);
 94   ht.uocTinhThoiGianCho(102);
 95   ht.uocTinhThoiGianCho(101);
 96   ht.uocTinhThoiGianCho(106);
 97   ht.uocTinhThoiGianCho(999);
 98
 99   cout << "\n-- 4. Chuyen khoa cap cuu uu tien --\n";
100   kq("Chuyen BN 103 (Ngoai)",     ht.chuyenKhoaCapCuuUuTien(103), &ht);
101   kq("Chuyen BN 103 lan nua",     ht.chuyenKhoaCapCuuUuTien(103), &ht);
102   kq("Chuyen BN 999 (khong co)",  ht.chuyenKhoaCapCuuUuTien(999), &ht);
103   ht.thongKePhongKham("Khoa Cap Cuu");
104   ht.thongKePhongKham("Ngoai");
105
106   cout << "\n-- 5. Luu file / doc lai (tinh ben vung) --\n";
107   const string tam = "data/demo_tam.csv";
108   try {
109   ht.luuCSV(tam);
110   cout << "  Da luu " << ht.kichThuoc() << " BN vao " << tam << "\n";
111   HeThongBenhVien ht2(50);
112   ht2.docCSV(tam);
113   cout << "  He thong MOI doc lai file: con " << ht2.kichThuoc() << " BN\n";
114   ht2.hienThiBangDienTu();
115   remove(tam.c_str());
116   } catch (const exception &e) {
117   cout << "  [LOI] " << e.what() << "\n";
118   }
119
120   cout << "\n-- 6. Goi so tu dong toan he thong (3 luot) --\n";
121   for (int i = 0; i < 3; i++) ht.goiSoTuDong();
122
123   cout << "\n-- 7. Goi so rieng phong Nhi --\n";
124   ht.goiSoTuDong("Nhi");
125   kq("Goi phong Nhi khi da het BN", ht.goiSoTuDong("Nhi"), &ht);
126   kq("Goi phong khong ton tai",     ht.goiSoTuDong("Mat"), &ht);
127
128   cout << "\n-- 8. Huy / goi theo ma --\n";
129   BenhNhan bn;
130   kq("Huy BN 106",                  ht.huyBenhNhan(106), &ht);
131   kq("Huy BN 106 lan nua",          ht.huyBenhNhan(106), &ht);
132   kq("Goi theo ma 101",             ht.goiTheoMa(101, bn), &ht);
133
134   cout << "\n";
135   ht.hienThiBangDienTu();
136   cout << "\n";
137   ht.hienThiLichSuGoi();
138   cout << "\nCon lai trong he thong: " << ht.kichThuoc() << " BN\n";
139  }
140
141  // ------------------------------------------------------------- menu
142  void inMenu() {
143   cout << "\n==================== CAREQUEUE UTC2 ====================\n"
144        << "  XEP HANG KHAM BENH & CAP CUU\n"
145        << "---------------------------------------------------------\n"
146        << "  1. Tiep nhan va phan luong benh nhan\n"
147        << "  2. Chuyen khoa cap cuu uu tien\n"
148        << "  3. Goi so tu dong (toan he thong)\n"
```

```
149         << "  4. Goi so theo phong\n"
150         << "  5. Goi so theo ma benh nhan\n"
151         << "  6. Huy phieu cua benh nhan\n"
152         << "  7. Uoc tinh thoi gian cho\n"
153         << "  8. Bang dien tu (danh sach cho)\n"
154         << "  9. Thong ke phong kham\n"
155         << " 10. Lich su goi so\n"
156         << " 11. Luu du lieu ra file\n"
157         << " 12. Doc du lieu tu file\n"
158         << " 13. Xoa toan bo du lieu\n"
159         << "  0. Thoat\n"
160         << "============================================================\n";
161 }
162
163 void chayMenu(HeThongBenhVien &ht, const string &file) {
164  while (true) {
165  inMenu();
166  int c = docSo("Chon chuc nang [0-13]: ", 0, 13);
167  cout << "\n";
168  if (c == 0) break;
169  switch (c) {
170  case 1: {
171  int ma = docSo("  Ma benh nhan (so duong): ", 1, 999999999);
172  string ten = docChuoiKhongRong("  Ho ten: ");
173  string phong = docChuoiKhongRong("  Phong kham (vd: Noi, Ngoai, Nhi): ");
174  int dut = docSo("  Do uu tien (1 = thuong ... 5 = khan nhat): ", 1, 5);
175  bool ok = ht.tiepNhanVaPhanLuong(ma, ten, phong, dut);
176  kq("Tiep nhan", ok, &ht);
177  if (ok) luuTuDong(ht, file);
178  break;
179  }
180  case 2: {
181  int ma = docSo("  Ma benh nhan can chuyen: ", 1, 999999999);
182  bool ok = ht.chuyenKhoaCapCuuUuTien(ma);
183  kq("Chuyen cap cuu", ok, &ht);
184  if (ok) luuTuDong(ht, file);
185  break;
186  }
187  case 3: {
188  if (ht.goiSoTuDong()) luuTuDong(ht, file);
189  else cout << "  " << ht.loiCuoi() << "\n";
190  break;
191  }
192  case 4: {
193  string p = docChuoiKhongRong("  Ten phong: ");
194  if (ht.goiSoTuDong(p)) luuTuDong(ht, file);
195  else cout << "  " << ht.loiCuoi() << "\n";
196  break;
197  }
198  case 5: {
199  int ma = docSo("  Ma benh nhan can goi: ", 1, 999999999);
200  BenhNhan bn;
201  if (ht.goiTheoMa(ma, bn)) {
202  cout << "[LOA THONG BAO] Moi BN STT " << bn.soThuTu << " - " << bn.ten
203       << " vao " << bn.phongKham << "!\n";
204  luuTuDong(ht, file);
```

```
205   } else {
206   cout << "  " << ht.loiCuoi() << "\n";
207   }
208   break;
209   }
210   case 6: {
211   int ma = docSo("  Ma benh nhan can huy: ", 1, 999999999);
212   bool ok = ht.huyBenhNhan(ma);
213   kq("Huy phieu", ok, &ht);
214   if (ok) luuTuDong(ht, file);
215   break;
216   }
217   case 7:
218   ht.uocTinhThoiGianCho(docSo("  Ma benh nhan: ", 1, 999999999));
219   break;
220   case 8:
221   ht.hienThiBangDienTu();
222   break;
223   case 9:
224   ht.thongKePhongKham(docChuoiKhongRong("  Ten phong: "));
225   break;
226   case 10:
227   ht.hienThiLichSuGoi();
228   break;
229   case 11:
230   try {
231   string f = docDong("  File luu [Enter = " + file + "]: ");
232   if (f.empty()) f = file;
233   ht.luuCSV(f);
234   cout << "  Da luu " << ht.kichThuoc() << " BN vao " << f << "\n";
235   } catch (const exception &e) {
236   cout << "  [LOI] " << e.what() << "\n";
237   }
238   break;
239   case 12:
240   try {
241   string f = docDong("  File doc [Enter = " + file + "]: ");
242   if (f.empty()) f = file;
243   ht.docCSV(f);
244   cout << "  Da doc " << ht.kichThuoc() << " BN tu " << f << "\n";
245   luuTuDong(ht, file);
246   } catch (const exception &e) {
247   cout << "  [LOI] " << e.what() << " (du lieu hien tai giu nguyen)\n";
248   }
249   break;
250   case 13: {
251   string x = docDong("  Xoa het du lieu? Go \"yes\" de xac nhan: ");
252   if (x == "yes") {
253   ht.datLai();
254   luuTuDong(ht, file);
255   cout << "  Da xoa toan bo du lieu.\n";
256   } else {
257   cout << "  Da huy.\n";
258   }
259   break;
260   }
```

```
 }
 }
}

void inTroGiup(const char *ten) {
 cout << "Cach dung: " << ten << " [--file <duong-dan.csv>] [--demo] [--help]\n"
      << "  (khong tham so)   Chay menu tuong tac, tu dong nap/luu " << FILE_MAC_DINH << "\n
"
      << "  --file <path>     Dung file CSV khac de luu tru\n"
      << "  --demo            Chay kich ban demo tu dong (khong dung file chinh)\n"
      << "  --help            Hien thi tro giup\n";
}

}  // namespace

int main(int argc, char **argv) {
 string file = FILE_MAC_DINH;
 bool demo = false;
 for (int i = 1; i < argc; i++) {
 string a = argv[i];
 if (a == "--demo") demo = true;
 else if (a == "--help" || a == "-h") { inTroGiup(argv[0]); return 0; }
 else if (a == "--file" && i + 1 < argc) file = argv[++i];
 else { cerr << "Tham so khong hop le: " << a << "\n"; inTroGiup(argv[0]); return 2; }
 }

 try {
 if (demo) {
 chayDemo();
 return 0;
 }

 // HeThongBenhVien lon (~vai tram KB) nen cap phat tren heap, tranh tran stack.
 HeThongBenhVien *ht = new HeThongBenhVien(50);
 try {
 error_code ec;
 if (filesystem::exists(file, ec)) {
 try {
 ht->docCSV(file);
 cout << "Da nap " << ht->kichThuoc() << " benh nhan tu " << file << "\n";
 } catch (const exception &e) {
 string hong = file + ".loi";
 error_code ec2;
 filesystem::copy_file(file, hong, filesystem::copy_options::overwrite_existing, ec2);
 cout << "[CANH BAO] Khong doc duoc " << file << ": " << e.what() << "\n"
      << "           Bat dau voi he thong rong; ban sao file loi: " << hong << "\n";
 }
 } else {
 cout << "Chua co file du lieu (" << file << ") - bat dau he thong moi.\n";
 }
 chayMenu(*ht, file);
 luuTuDong(*ht, file);
 cout << "Tam biet!\n";
 } catch (const HetDuLieuNhap &) {
 luuTuDong(*ht, file);
 cout << "\n(Het du lieu nhap) Da luu va thoat.\n";
```

```
 } catch (...) {
 delete ht;
 throw;
 }
 delete ht;
 } catch (const HetDuLieuNhap &) {
 cout << "\n(Het du lieu nhap)\n";
 } catch (const exception &e) {
 cerr << "Loi nghiem trong: " << e.what() << "\n";
 return 1;
 }
 return 0;
}
```

## tests/test_all.cpp

```
// Bo kiem thu tu dong (khong can thu vien ngoai). Chay: make test
#include <cstdio>
#include <fstream>
#include <sstream>
#include "HeThongBenhVien.h"

static int tong = 0, dat = 0;

#define CHECK(id, moTa, cond) do { tong++; if (cond) { dat++; printf("  [PASS] %s %s\n", id, moTa); } \
    else printf("  [FAIL] %s %s   (dong %d)\n", id, moTa, __LINE__); } while (0)

static BenhNhan mk(int ma, int ut, int stt) {
 BenhNhan b; b.maBenhNhan = ma; b.doUuTien = ut; b.soThuTu = stt; b.ten = "BN"; b.phongKham = "P";
 return b;
}

// Chay f() trong khi tat cout (cac ham he thong in thong bao ra man hinh)
template <class F> static void im(F f) {
 ostringstream sink;
 streambuf *cu = cout.rdbuf(sink.rdbuf());
 f();
 cout.rdbuf(cu);
}
static string layCout(void (*f)(HeThongBenhVien&), HeThongBenhVien &h) {
 ostringstream sink;
 streambuf *cu = cout.rdbuf(sink.rdbuf());
 f(h);
 cout.rdbuf(cu);
 return sink.str();
}

static void testHangDoi() {
 puts("\n== HangDoi (Queue mang vong) ==");
 HangDoi q; BenhNhan b;
 CHECK("Q01", "hang moi rong, lay that bai", q.laRong() && !q.lay(b) && !q.xemDau(b));
 for (int i = 1; i <= 3; i++) q.them(mk(i, 1, i));
 q.lay(b);
```

```
 CHECK("Q02", "FIFO: lay phan tu vao truoc", b.maBenhNhan == 1 && q.kichThuoc() == 2);
 HangDoi day;
 bool tatCaThem = true;
 for (int i = 1; i <= MAX_HANG_DOI; i++) tatCaThem = tatCaThem && day.them(mk(i, 1, i));
 CHECK("Q03", "them du MAX phan tu, them nua bi tu choi", tatCaThem && day.laDay() && !day.them(mk(999, 1, 999)));
 // vong: lay 30 roi them 30 -> con tro dau quay vong
 for (int i = 0; i < 30; i++) day.lay(b);
 bool vong = true;
 for (int i = 0; i < 30; i++) vong = vong && day.them(mk(1000 + i, 1, 1000 + i));
 day.xemTaiViTri(MAX_HANG_DOI - 1, b);
 CHECK("Q04", "quay vong con tro (wrap-around) dung thu tu", vong && b.maBenhNhan == 1029 && day.kichThuoc() == MAX_HANG_DOI);
 HangDoi t;
 for (int i = 1; i <= 5; i++) t.them(mk(i, 1, i));
 CHECK("Q05", "layTheoMa o giua giu nguyen thu tu con lai", t.layTheoMa(3, b) && t.kichThuoc() == 4);
 t.xemTaiViTri(2, b);
 CHECK("Q06", "sau khi xoa 3, vi tri 2 la ma 4", b.maBenhNhan == 4);
 CHECK("Q07", "layTheoMa ma khong ton tai -> false", !t.layTheoMa(77, b) && !t.xemTaiViTri(9, b) && !t.xemTaiViTri(-1, b));
}

static void testHangUuTien() {
 puts("\n== HangUuTien (Priority Queue danh sach lien ket) ==");
 HangUuTien pq; BenhNhan b;
 CHECK("P01", "hang moi rong", pq.laRong() && !pq.lay(b) && !pq.xemDau(b));
 pq.them(mk(1, 2, 1)); pq.them(mk(2, 5, 2)); pq.them(mk(3, 3, 3)); pq.them(mk(4, 5, 4));
 int th[4]; for (int i = 0; i < 4; i++) { pq.lay(b); th[i] = b.maBenhNhan; }
 CHECK("P02", "lay theo uu tien giam dan, cung muc theo FIFO (2,4,3,1)", th[0] == 2 && th[1] == 4 && th[2] == 3 && th[3] == 1);
 pq.them(mk(10, 3, 1)); pq.them(mk(11, 3, 2)); pq.them(mk(12, 4, 3));
 CHECK("P03", "layTheoMa phan tu giua / dau / cuoi", pq.layTheoMa(10, b) && pq.layTheoMa(12, b) && pq.layTheoMa(11, b) && pq.laRong());
 CHECK("P04", "layTheoMa tren hang rong -> false", !pq.layTheoMa(10, b));
 HangUuTien day; bool ok = true;
 for (int i = 1; i <= MAX_HANG_DOI; i++) ok = ok && day.them(mk(i, 2, i));
 CHECK("P05", "day MAX phan tu thi tu choi them", ok && !day.them(mk(999, 5, 999)) && day.kichThuoc() == MAX_HANG_DOI);
 CHECK("P06", "xemTaiViTri hop le / bien", day.xemTaiViTri(0, b) && b.maBenhNhan == 1 && !day.xemTaiViTri(MAX_HANG_DOI, b));
}

static void testDeque() {
 puts("\n== HangDoiHaiDau (Deque danh sach lien ket doi) ==");
 HangDoiHaiDau d; BenhNhan b;
 CHECK("D01", "deque rong", d.laRong() && !d.layDau(b) && !d.layCuoi(b));
 d.themCuoi(mk(2, 1, 2)); d.themCuoi(mk(3, 1, 3)); d.themDau(mk(1, 1, 1));
 d.xemTaiViTri(0, b); int a0 = b.maBenhNhan; d.xemTaiViTri(2, b); int a2 = b.maBenhNhan;
 CHECK("D02", "themDau/themCuoi cho thu tu 1,2,3", a0 == 1 && a2 == 3 && d.kichThuoc() == 3);
 d.layCuoi(b); int c = b.maBenhNhan; d.layDau(b);
 CHECK("D03", "layCuoi=3, layDau=1", c == 3 && b.maBenhNhan == 1 && d.kichThuoc() == 1);
 d.layCuoi(b); d.themDau(mk(7, 1, 7));
 CHECK("D04", "lay het roi them lai van hoat dong (con tro dau/cuoi)", d.kichThuoc() == 1 && d.layDau(b) && b.maBenhNhan == 7 && d.laRong());
```

```
 84 }
 85
 86 static void testHeThong() {
 87  puts("\n== HeThongBenhVien (nghiep vu) ==");
 88  {
 89  HeThongBenhVien h(50);
 90  CHECK("H01", "tiep nhan hop le", h.tiepNhanVaPhanLuong(1, "A", "Noi", 1));
 91  CHECK("H02", "trung ma bi tu choi", !h.tiepNhanVaPhanLuong(1, "B", "Noi", 1));
 92  CHECK("H03", "ma <= 0 bi tu choi", !h.tiepNhanVaPhanLuong(0, "B", "Noi", 1) && !h.tiepNhan
    VaPhanLuong(-5, "B", "Noi", 1));
 93  CHECK("H04", "uu tien ngoai 1..5 bi tu choi", !h.tiepNhanVaPhanLuong(2, "B", "Noi", 0) &&
    !h.tiepNhanVaPhanLuong(2, "B", "Noi", 6));
 94  CHECK("H05", "ten / phong rong hoac toan khoang trang bi tu choi",
 95        !h.tiepNhanVaPhanLuong(2, "", "Noi", 1) && !h.tiepNhanVaPhanLuong(2, "B", "", 1) &&
    !h.tiepNhanVaPhanLuong(2, "   ", "Noi", 1));
 96  CHECK("H06", "ten chua dau phay / xuong dong bi tu choi", !h.tiepNhanVaPhanLuong(2, "A,B",
     "Noi", 1) && !h.tiepNhanVaPhanLuong(2, "A\nB", "Noi", 1));
 97  CHECK("H07", "ten qua 50 ky tu bi tu choi", !h.tiepNhanVaPhanLuong(2, string(51, 'x'), "No
    i", 1) && h.tiepNhanVaPhanLuong(2, string(50, 'x'), "Noi", 1));
 98  CHECK("H08", "so luong khong doi khi tiep nhan that bai", h.kichThuoc() == 2);
 99  }
100  {
101  HeThongBenhVien h(3);
102  h.tiepNhanVaPhanLuong(1, "A", "Noi", 1); h.tiepNhanVaPhanLuong(2, "B", "Noi", 2); h.tiepNh
    anVaPhanLuong(3, "C", "Noi", 3);
103  CHECK("H09", "vuot suc chua he thong bi tu choi", !h.tiepNhanVaPhanLuong(4, "D", "Noi", 1)
     && h.kichThuoc() == 3 && !h.loiCuoi().empty());
104  }
105  {
106  HeThongBenhVien h(100);
107  bool ok = true;
108  for (int i = 0; i < SO_PHONG_TOI_DA; i++) ok = ok && h.tiepNhanVaPhanLuong(i + 1, "BN", "P
    " + to_string(i), 1);
109  CHECK("H10", "toi da 10 phong, phong thu 11 bi tu choi", ok && !h.tiepNhanVaPhanLuong(99,
    "BN", "PX", 1));
110  }
111  {
112  HeThongBenhVien h(50); BenhNhan b;
113  h.tiepNhanVaPhanLuong(101, "A", "Noi", 1);
114  h.tiepNhanVaPhanLuong(102, "B", "Noi", 3);
115  h.tiepNhanVaPhanLuong(103, "C", "Ngoai", 1);
116  h.tiepNhanVaPhanLuong(104, "D", "Noi", 5);
117  h.tiepNhanVaPhanLuong(105, "E", "Noi", 3);
118  h.tiepNhanVaPhanLuong(106, "F", "Noi", 1);
119  h.layBenhNhanTiepTheo("Noi", b); int g1 = b.maBenhNhan;
120  h.layBenhNhanTiepTheo("Noi", b); int g2 = b.maBenhNhan;
121  h.layBenhNhanTiepTheo("Noi", b); int g3 = b.maBenhNhan;
122  h.layBenhNhanTiepTheo("Noi", b); int g4 = b.maBenhNhan;
123  CHECK("H11", "goi theo phong: UT5, UT3 (FIFO 102 truoc 105), roi thuong FIFO", g1 == 104 &
    & g2 == 102 && g3 == 105 && g4 == 101);
124  CHECK("H12", "goi het phong -> that bai co ly do", h.layBenhNhanTiepTheo("Noi", b) && h.la
    yBenhNhanTiepTheo("Noi", b) == false && !h.loiCuoi().empty());
125  CHECK("H13", "goi phong khong ton tai -> that bai", !h.layBenhNhanTiepTheo("Mat", b));
126  }
127  {
128  HeThongBenhVien h(50); BenhNhan b;
```

```
 h.tiepNhanVaPhanLuong(1, "Thuong", "Noi", 1);   // stt 1
 h.tiepNhanVaPhanLuong(2, "Ngoai", "Ngoai", 1);  // stt 2
 h.tiepNhanVaPhanLuong(3, "Nhi", "Nhi", 2);      // stt 3
 h.tiepNhanVaPhanLuong(4, "Khan", "Noi", 4);     // stt 4
 im([&] { h.chuyenKhoaCapCuuUuTien(2); });
 h.layBenhNhanTiepTheo("", b); int a = b.maBenhNhan;
 h.layBenhNhanTiepTheo("", b); int c = b.maBenhNhan;
 h.layBenhNhanTiepTheo("", b); int d = b.maBenhNhan;
 h.layBenhNhanTiepTheo("", b); int e = b.maBenhNhan;
 CHECK("H14", "goi tu dong toan he: cap cuu(2) -> UT4 -> UT2 -> thuong", a == 2 && c == 4 &
& d == 3 && e == 1);
 CHECK("H15", "het BN -> goi tu dong that bai", !h.goiSoTuDong() && h.laRong() && h.kichThu
oc() == 0);
 }
 {
 HeThongBenhVien h(50);
 h.tiepNhanVaPhanLuong(1, "A", "Noi", 1);
 bool ok1, ok2, ok3, ok4;
 im([&] {
 ok1 = h.chuyenKhoaCapCuuUuTien(1);
 ok2 = h.chuyenKhoaCapCuuUuTien(1);
 ok3 = h.chuyenKhoaCapCuuUuTien(77);
 });
 CHECK("H16", "chuyen khoa cap cuu: lan 1 ok, lan 2 va ma la that bai", ok1 && !ok2 && !ok3
);
 ok4 = h.thoiGianChoDuKien(1) == 0;
 CHECK("H17", "BN cap cuu o dau hang -> cho 0 phut", ok4);
 }
 {
 HeThongBenhVien h(50); BenhNhan b;
 h.tiepNhanVaPhanLuong(1, "A", "Noi", 1);
 h.tiepNhanVaPhanLuong(2, "B", "Noi", 1);
 h.tiepNhanVaPhanLuong(3, "C", "Noi", 3);
 CHECK("H18", "thoi gian cho: UT truoc, thuong sau (0,10,20 phut)",
       h.thoiGianChoDuKien(3) == 0 && h.thoiGianChoDuKien(1) == 10 && h.thoiGianChoDuKien(2
) == 20);
 CHECK("H19", "thoi gian cho ma khong ton tai = -1", h.thoiGianChoDuKien(404) == -1);
 CHECK("H20", "huy BN: ok 1 lan, lan sau that bai", h.huyBenhNhan(2) && !h.huyBenhNhan(2) &
& h.kichThuoc() == 2);
 CHECK("H21", "huy khong ghi vao lich su, goi theo ma thi co",
       h.goiTheoMa(1, b) && b.maBenhNhan == 1 && !h.goiTheoMa(1, b));
 BenhNhan ls[10];
 CHECK("H22", "lich su goi chi co 1 BN (ma 1)", h.lietKeLichSuGoi(ls, 10) == 1 && ls[0].maB
enhNhan == 1);
 CHECK("H28", "lich su voi mang NULL / suc chua khong hop le -> 0", h.lietKeLichSuGoi(NULL,
 5) == 0 && h.lietKeLichSuGoi(ls, 0) == 0);
 }
 {
 HeThongBenhVien h(100); BenhNhan b;
 for (int i = 1; i <= 100; i++) h.tiepNhanVaPhanLuong(i, "BN", "Noi", 1);
 for (int i = 1; i <= 100; i++) h.layBenhNhanTiepTheo("Noi", b);
 BenhNhan ls[100];
 int n = h.lietKeLichSuGoi(ls, 100);
 CHECK("H23", "lich su day 100: giu du 100, BN cu nhat con la 1", n == 100 && ls[0].maBenhN
han == 1 && ls[99].maBenhNhan == 100);
 h.tiepNhanVaPhanLuong(101, "BN", "Noi", 1);
```

```
 h.layBenhNhanTiepTheo("Noi", b);
 n = h.lietKeLichSuGoi(ls, 100);
 CHECK("H24", "goi them 1 -> lich su van 100, BN 1 bi day ra (Deque layDau)", n == 100 && ls[0].maBenhNhan == 2 && ls[99].maBenhNhan == 101);
 }
 {
 HeThongBenhVien h(50);
 h.tiepNhanVaPhanLuong(1, "A", "Noi", 1);
 h.tiepNhanVaPhanLuong(2, "B", "Noi", 4);
 h.tiepNhanVaPhanLuong(3, "C", "Nhi", 2);
 BenhNhan ds[10];
 int n = h.lietKeThuTuCho(ds, 10);
 CHECK("H25", "lietKeThuTuCho sap xep toan he theo uu tien", n == 3 && ds[0].maBenhNhan == 2 && ds[1].maBenhNhan == 3 && ds[2].maBenhNhan == 1);
 CHECK("H26", "lietKeThuTuCho tham so xau (NULL / 0) an toan", h.lietKeThuTuCho(NULL, 5) == 0 && h.lietKeThuTuCho(ds, 0) == 0 && h.lietKeThuTuCho(ds, 2) == 2);
 }
 {
 HeThongBenhVien h(50);
 h.tiepNhanVaPhanLuong(1, "A", "Noi", 1);
 string s = layCout([](HeThongBenhVien &x) { x.hienThiBangDienTu(); }, h);
 string s2 = layCout([](HeThongBenhVien &x) { x.thongKePhongKham("Khong co"); }, h);
 CHECK("H27", "bang dien tu / thong ke phong la khong crash, co noi dung", s.find("Nguyen") == string::npos && s.find("A") != string::npos && s2.find("khong ton tai") != string::npos);
 }
}

static string docFile(const string &p) {
 ifstream f(p.c_str()); ostringstream s; s << f.rdbuf(); return s.str();
}
static void ghiFile(const string &p, const string &c) {
 ofstream f(p.c_str(), ios::trunc); f << c;
}
static bool nemLoi(HeThongBenhVien &h, const string &p) {
 try { h.docCSV(p); } catch (const runtime_error &) { return true; } catch (...) { return false; }
 return false;
}

static void testLuuTru() {
 puts("\n== Luu / doc CSV (tinh ben vung) ==");
 const string f = "data/test_tmp.csv";
 HeThongBenhVien a(50);
 a.tiepNhanVaPhanLuong(101, "Nguyen Van A", "Noi", 1);
 a.tiepNhanVaPhanLuong(102, "Tran Thi B", "Noi", 3);
 a.tiepNhanVaPhanLuong(103, "Le Van C", "Ngoai", 1);
 a.tiepNhanVaPhanLuong(104, "Pham Thi D", "Noi", 5);
 a.tiepNhanVaPhanLuong(105, "Hoang Van E", "Nhi", 2);
 a.tiepNhanVaPhanLuong(106, "Do Thi G", "Noi", 1);
 im([&] { a.chuyenKhoaCapCuuUuTien(103); a.goiSoTuDong(); a.goiSoTuDong("Nhi"); });
 bool luuOk = true;
 try { a.luuCSV(f); } catch (...) { luuOk = false; }
 CHECK("F01", "luu file thanh cong, khong de lai file .tmp", luuOk && !ifstream((f + ".tmp").c_str()));

```

```
226  HeThongBenhVien b(50);
227  bool docOk = true;
228  try { b.docCSV(f); } catch (...) { docOk = false; }
229  CHECK("F02", "doc file vao he thong moi thanh cong", docOk && b.kichThuoc() == a.kichThuoc(
     ));
230  string sa = layCout([](HeThongBenhVien &x) { x.hienThiBangDienTu(); x.hienThiLichSuGoi(); }
     , a);
231  string sb = layCout([](HeThongBenhVien &x) { x.hienThiBangDienTu(); x.hienThiLichSuGoi(); }
     , b);
232  // thu tu phong co the khac vi cap cuu chuyen -> so sanh theo goi so lan luot
233  BenhNhan x, y; bool giong = true;
234  for (int i = 0; i < 6; i++) {
235  bool u, v;
236  im([&] { u = a.layBenhNhanTiepTheo("", x); v = b.layBenhNhanTiepTheo("", y); });
237  if (u != v || (u && (x.maBenhNhan != y.maBenhNhan || x.soThuTu != y.soThuTu))) giong = fal
     se;
238  }
239  CHECK("F03", "thu tu goi sau khi nap giong het ban goc", giong);
240  (void)sa; (void)sb;
241
242  // STT tiep tuc dung sau khi nap
243  HeThongBenhVien c(50);
244  c.docCSV(f);
245  c.tiepNhanVaPhanLuong(200, "Moi", "Noi", 1);
246  BenhNhan ds[10];
247  int n = c.lietKeThuTuCho(ds, 10), maxStt = 0;
248  for (int i = 0; i < n; i++) if (ds[i].soThuTu > maxStt) maxStt = ds[i].soThuTu;
249  CHECK("F04", "sau khi nap, STT tiep tuc (BN moi nhan STT 7)", maxStt == 7);
250  BenhNhan ls[100];
251  CHECK("F05", "lich su goi duoc khoi phuc (2 luot)", c.lietKeLichSuGoi(ls, 100) == 2);
252
253  // roundtrip lap lai khong doi noi dung
254  c.luuCSV(f + "2"); HeThongBenhVien d(50); d.docCSV(f + "2"); d.luuCSV(f + "3");
255  CHECK("F06", "luu -> doc -> luu cho file giong nhau (on dinh)", docFile(f + "2") == docFile
     (f + "3"));
256
257  HeThongBenhVien e(50);
258  CHECK("F07", "file khong ton tai -> nem runtime_error", nemLoi(e, "data/khong_co.csv"));
259  ghiFile(f, "");
260  CHECK("F08", "file rong -> nem loi", nemLoi(e, f));
261  ghiFile(f, "NOT A CAREQUEUE FILE\n");
262  CHECK("F09", "sai tieu de -> nem loi", nemLoi(e, f));
263  ghiFile(f, "CAREQUEUE,1\nSTT,5\nQ,abc,A,Noi,1,1\n");
264  CHECK("F10", "ma khong phai so -> nem loi", nemLoi(e, f));
265  ghiFile(f, "CAREQUEUE,1\nSTT,5\nQ,1,A,Noi,9,1\n");
266  CHECK("F11", "uu tien ngoai khoang -> nem loi", nemLoi(e, f));
267  ghiFile(f, "CAREQUEUE,1\nSTT,5\nQ,1,A,Noi,1,1\nQ,1,B,Noi,1,2\n");
268  CHECK("F12", "trung ma benh nhan -> nem loi", nemLoi(e, f));
269  ghiFile(f, "CAREQUEUE,1\nSTT,5\nQ,1,A,Noi,1\n");
270  CHECK("F13", "thieu truong -> nem loi", nemLoi(e, f));
271  ghiFile(f, "CAREQUEUE,1\nQ,1,A,Noi,1,1\n");
272  CHECK("F14", "thieu dong STT -> nem loi", nemLoi(e, f));
273  ghiFile(f, "CAREQUEUE,1\nSTT,1\nQ,1,A,Noi,1,5\n");
274  CHECK("F15", "STT benh nhan lon hon STT da cap -> nem loi", nemLoi(e, f));
275  ghiFile(f, "CAREQUEUE,1\nSTT,1\nX,1,A\n");
276  CHECK("F16", "loai ban ghi la -> nem loi", nemLoi(e, f));
```

```
277
278   // nap loi khong lam hong du lieu dang co (strong guarantee)
279   HeThongBenhVien g(50);
280   g.tiepNhanVaPhanLuong(1, "Giu", "Noi", 1);
281   ghiFile(f, "CAREQUEUE,1\nSTT,3\nQ,2,A,Noi,1,1\nQ,3,B,Noi,99,2\n");
282   bool loi = nemLoi(g, f);
283   CHECK("F17", "nap file hong that bai -> du lieu dang co giu nguyen", loi && g.kichThuoc() ==
      = 1 && g.thoiGianChoDuKien(1) == 0);
284
285   // Windows CRLF
286   ghiFile(f, "CAREQUEUE,1\r\nSTT,1\r\nR,Noi\r\nQ,1,A,Noi,1,1\r\n");
287   HeThongBenhVien h(50);
288   bool crlf = true; try { h.docCSV(f); } catch (...) { crlf = false; }
289   CHECK("F18", "file dinh dang CRLF (Windows) doc duoc", crlf && h.kichThuoc() == 1);
290
291   CHECK("F19", "duong dan rong -> luuCSV nem loi", [] { HeThongBenhVien z; try { z.luuCSV("")
      ; } catch (const runtime_error &) { return true; } return false; }());
292   CHECK("F20", "datLai xoa sach he thong", [&] { h.datLai(); return h.laRong() && h.soPhongHi
      enCo() == 0; }());
293
294   for (const char *s : {"", "2", "3"}) remove((f + s).c_str());
295 }
296
297 int main() {
298   puts("=== BO KIEM THU CAREQUEUE ===");
299   testHangDoi();
300   testHangUuTien();
301   testDeque();
302   testHeThong();
303   testLuuTru();
304   printf("\nKET QUA: %d/%d dat\n", dat, tong);
305   return dat == tong ? 0 : 1;
306 }
```